



Ciencias de la
Computación

Ceibal Ceibal

Fundamentos CSS

INICIAR

- ≡ ¿Qué es una hoja de estilos?
- ≡ Selectores
- ≡ Estilos en Cascada y Herencia
- ≡ Tamaños y distancias
- ≡ Modelo de caja (Box Model)
- ≡ Resumen

PROPIEDADES CSS

- ≡ background-color
- ≡ Tipografía (font-family, font-size)
- ≡ color
- ≡ text-align
- ≡ text-decoration
- ≡ border
- ≡ border-bottom, border-left, border-right, border-top
- ≡ border color
- ≡ Bordes redondeados
- ≡ Bordes-ancho
- ≡ display
- ≡ float
- ≡ Modelo de caja Básico (height, margin, padding, width)
- ≡ Pseudo-clases

UNIFICANDO CONCEPTOS

- ≡ Construyendo Menús de Navegación
- ≡ Construyendo Galería de imágenes
- ≡ Construyendo un Formulario con estilo

¿Qué es una hoja de estilos?



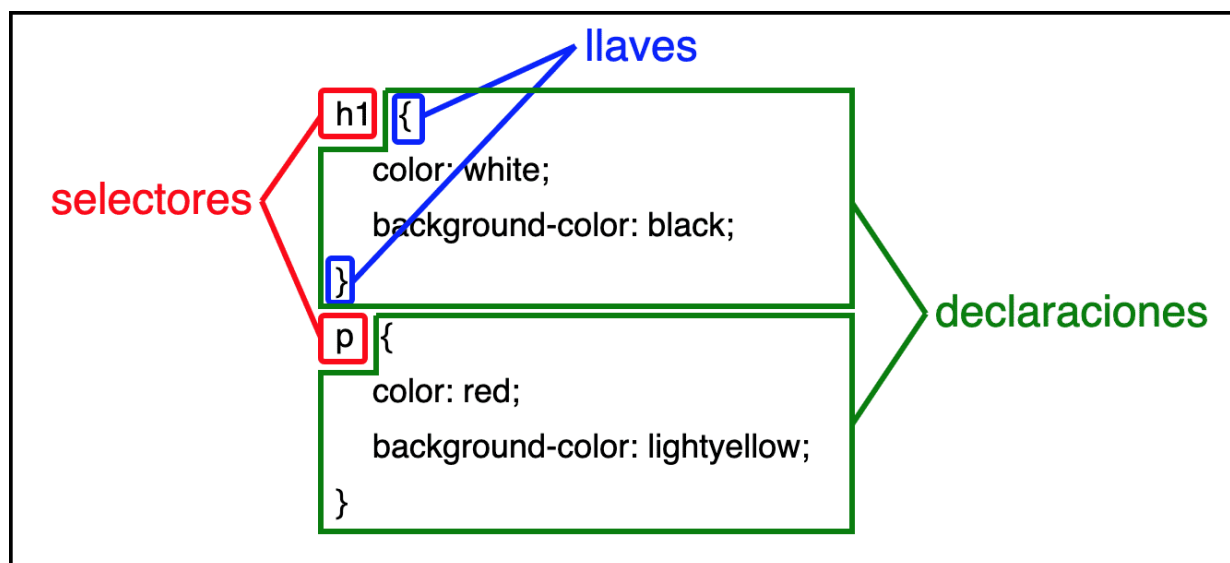
¿Qué es CSS?

En las primeras versiones de HTML, el código fuente de una página web contenía tanto la información (el contenido) como la forma de representarse (el diseño o formato).

Actualmente, estos dos aspectos se pueden separar. La página web (el documento html) puede solo contener información, mientras que el formato se define generalmente en las llamadas hojas de estilo (en inglés, **CSS**, **Cascading Style Sheets**, es decir, **Hojas de Estilo en Cascada**).

La sintaxis básica de CSS implica tres conceptos clave:

Selector y un bloque de declaración. El bloque de declaración empieza y acaba con llaves { }.



Selectores: Nos permiten decir a qué elemento de HTML queremos aplicarle estilos.

Propiedades: Indican qué estilo queremos aplicar, como color, background, font-family, font-size, etc. Estas suelen ser palabras en inglés.

Valores: Es el valor que queremos asignar a esa propiedad, por ejemplo, red para el color, yellow para el fondo, Arial para la fuente.



Reglas sintácticas

✓ Cada declaración CSS está formada por un par **propiedad: valor;**

✓ No se ve afectado por el espacio en blanco. Las propiedades se pueden escribir de corrido o una debajo de la otra.

✓ Siempre que la propiedad represente un número, el valor debe indicar en qué unidad se expresa.

¿Cómo incorporar CSS en nuestra página?

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>

  <p style="color: red;">Párrafo coloreado a través del ATRIBUTO STYLE</p>

  <p>Párrafo sin colorear</p>

</body>
</html>
```

Párrafo coloreado a través del ATRIBUTO STYLE

Párrafo sin colorear

En el html, a través del **atributo style**, afectando solamente al elemento en el que es incluido. Esta práctica es muy poco recomendable, consiste en usar el atributo style directamente en las etiquetas HTML para "parches" específicos, o pruebas. Se hace difícil mantenerlo, y es considerado una mala práctica.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    span {color:red;}
  </style>
</head>
<body>

  <p><span>Este</span> span está coloreado con una ETIQUETA STYLE</p>

  <p>Y <span>este</span> span también es coloreado con la ETIQUETA STYLE</p>

</body>
</html>

```

Este span está coloreado con una ETIQUETA STYLE

Y este span también es coloreado con la ETIQUETA STYLE

En el html, a través de la **etiqueta style**, afectando los elementos seleccionados en toda la página web.

Forma **interna**: se agrega una etiqueta **<style>** dentro de la etiqueta **<head>**. Puede estar en **<body>**, pero sería más desprolijo. Esta práctica no es recomendable.

```

index.html X # estilos.css
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Document</title>
6   <link rel="stylesheet" type="text/css" href="estilos.css"/>
7 </head>
8 <body>
9
10  <p><span>Este</span> span está coloreado desde la HOJA DE ESTILOS</p>
11
12  <p>Y <span>este</span> span también es coloreado desde la HOJA DE ESTILOS</p>
13
14 </body>
15 </html>

```

Este span está coloreado desde la HOJA DE ESTILOS

Y este span también es coloreado desde la HOJA DE ESTILOS

```

# estilos.css X
1 span {
2   color: red;
3 }

```

Recuerda el uso de rutas relativas y absolutas.

En la **hoja de estilos**, de la siguiente forma:

- En la página web (archivo .html) se escribe qué hoja (u hojas) de estilo queremos utilizar a través de la etiqueta link.
- En la hoja de estilos (archivo .css)

se escribe cómo queremos que sea el estilo de presentación de las etiquetas (color, tamaño, fuente, bordes, márgenes, posición, etc).

El archivo CSS (ej: styles.css) se crea dentro de la carpeta css. Es convención llamarlo styles.css. Para vincular el archivo CSS externo al HTML, se utiliza la etiqueta **<link>** dentro del **<head> del HTML**.

La etiqueta **<link>** debe especificar la relación (**rel="stylesheet"**) y la referencia (**href**) a la ubicación del archivo CSS.

 SINTAXIS HTML:

<head>

<link rel="stylesheet" href="styles.css">

</head>

Es crucial especificar la ruta correcta al archivo CSS, incluyendo el nombre de la carpeta si está dentro de una, por ejemplo:

<link rel="stylesheet" href="css/styles.css">.

Si el archivo CSS estuviera en la misma carpeta que el HTML, solo sería necesario poner el nombre del archivo (ej: styles.css), pero la convención es separarlos en carpetas.

<link rel="stylesheet" href="styles.css">.

Una vez vinculado de manera externa, el HTML aplicará los estilos definidos en el archivo CSS, logrando el mismo resultado que si se hiciera de forma interna, pero de manera más organizada.

Las ventajas de utilizar hojas de estilo son muchas, sobre todo **permiten hacer un diseño consistente y fácil de modificar.**

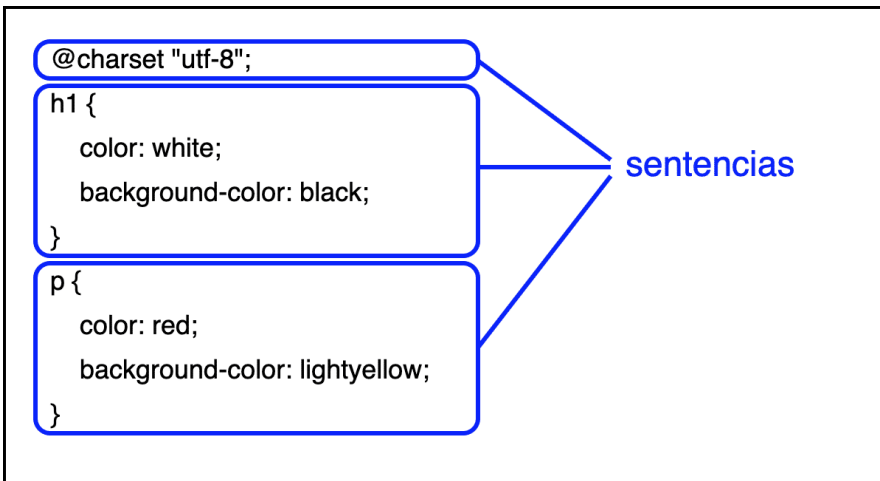
Si varias páginas web hacen referencia a la misma hoja de estilo, para cambiar la apariencia de todas las páginas es suficiente con hacer los cambios en un único lugar, en la hoja de estilo.

Sintaxis de las hojas de estilo

Una hoja de estilo es un fichero de texto plano (sin formato) en el que se define el aspecto de las etiquetas de una página web:

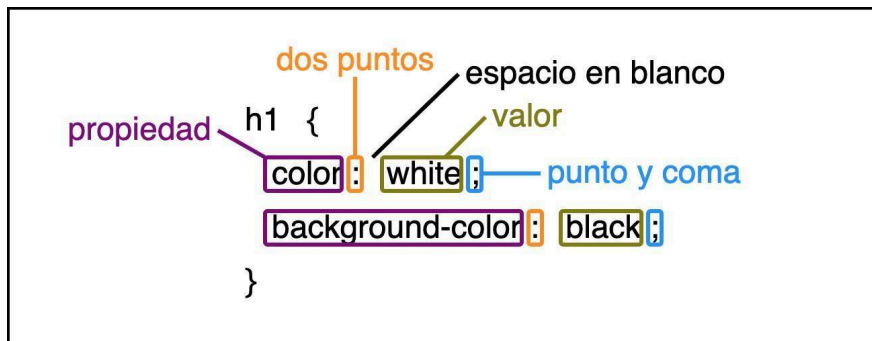
```
@charset "utf-8";  
h1 {  
    color: white;  
    background-color: black;  
}  
p {  
    color: red;  
    background-color: lightyellow;  
}
```

Una hoja de estilo está formada por una o varias sentencias:



Cada declaración está formada por una o varias propiedades y sus valores correspondientes. Las propiedades van separadas entre sí por puntos y comas.

El valor (o valores) van separados de las propiedades por dos puntos.



Un error bastante habitual es escribir los caracteres '`< >`' en el selector de la hoja de estilo.



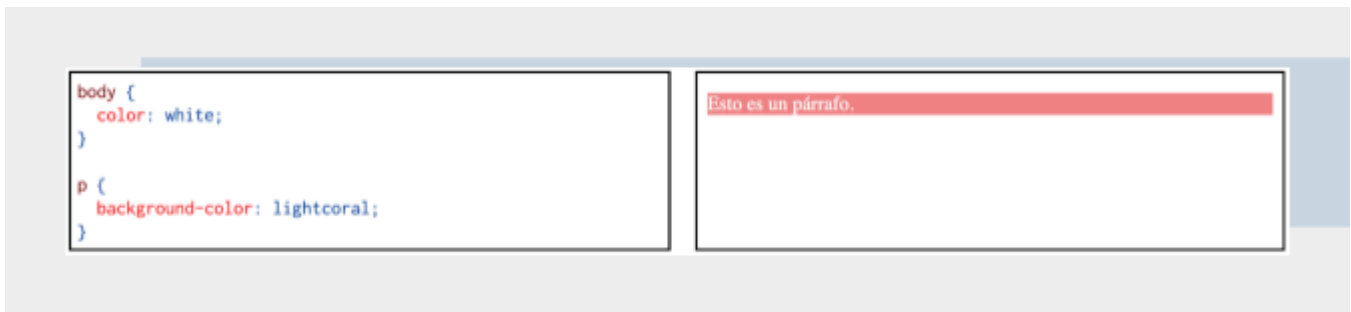
El elemento no es modificado ya que el selector tiene incluidos los caracteres '`< >`'

Estilos en cascada

El formato final de cualquier elemento depende de la combinación de todas las propiedades que afectan al elemento.

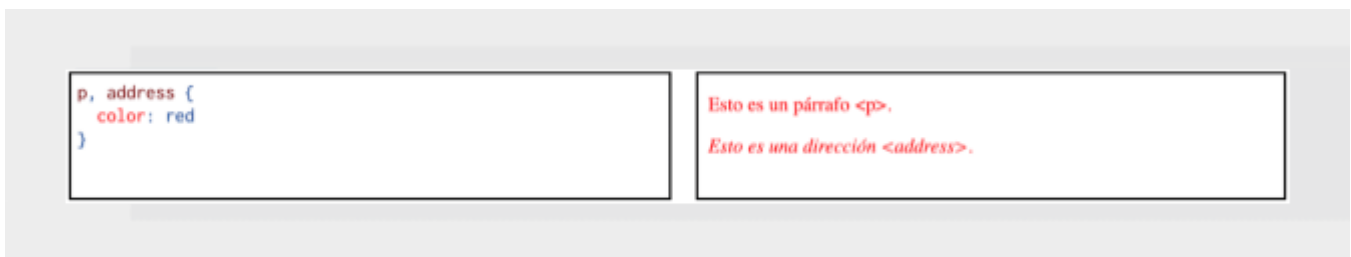
La denominación de hojas de estilo en cascada hace referencia precisamente a este hecho.

Por ejemplo, una de esas reglas es que a un elemento le afectan todas las propiedades de los elementos en los que está incluido, como muestra el ejemplo siguiente, en el que el color de texto del párrafo `<p>` viene determinado por la propiedad establecida para el elemento `<body>`:



Definir varios selectores simultáneamente

Se pueden definir varios selectores simultáneamente escribiendo los selectores separados por comas, como muestran los siguientes ejemplos:



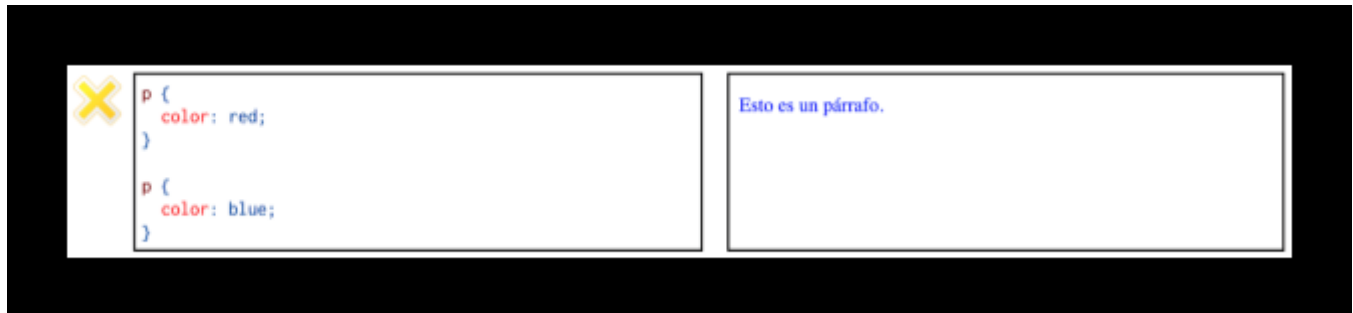
Lo que no es correcto es definir varias propiedades simultáneamente, como muestran los siguientes ejemplos:



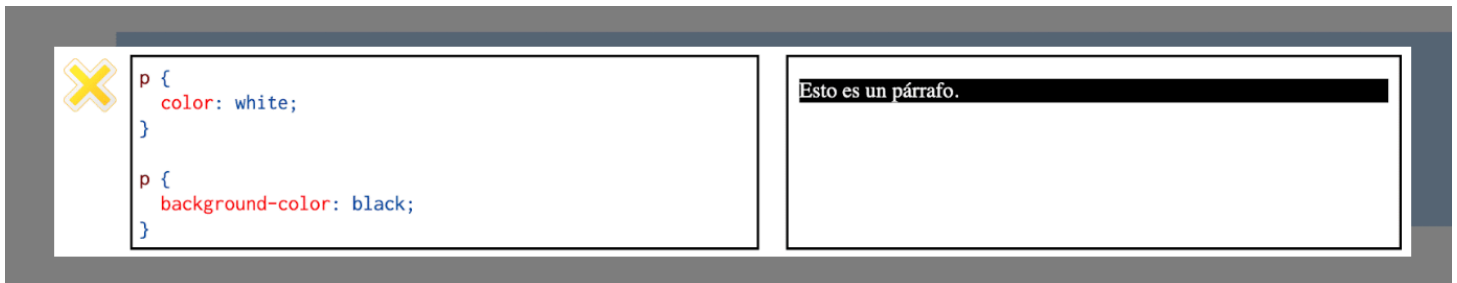
Repetición de selectores

Aunque no es incorrecto, no se aconseja repetir selectores, ya que distribuye la información en varios lugares de la hoja de estilo y eso dificulta la lectura y interpretación de la hoja de estilo a los lectores humanos.

Si se repiten las propiedades, el navegador aplica la propiedad que aparece en último lugar, como muestra el siguiente ejemplo:

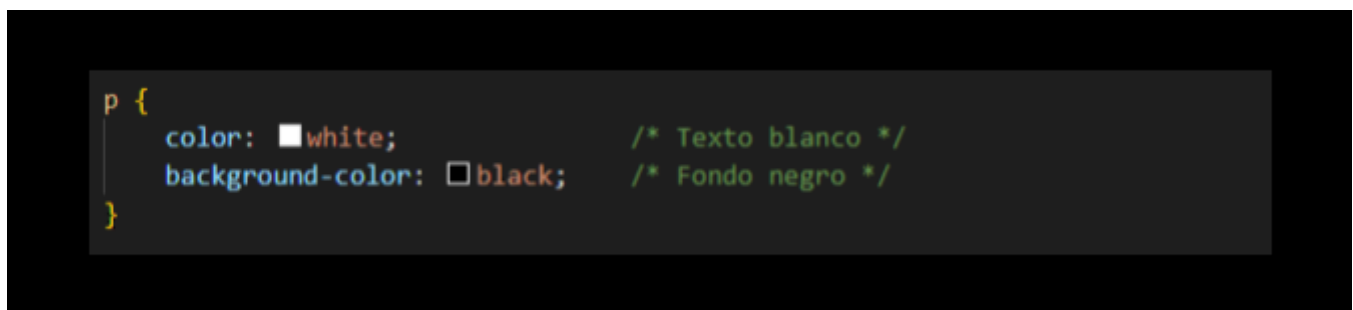


Si no se repiten las propiedades, el navegador aplica todas las propiedades, como muestra el siguiente ejemplo:



Comentarios en las hojas de estilo

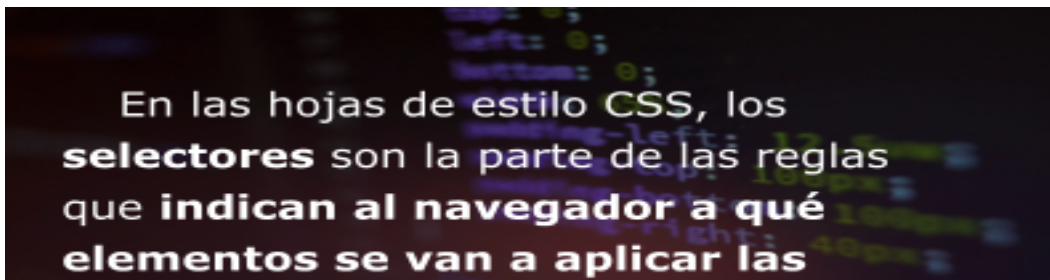
Una hoja de estilo puede contener comentarios. Los delimitadores del comentario son /* ... */ (como en el lenguaje de programación C). Los comentarios pueden extenderse varias líneas, como ilustra el siguiente ejemplo:



Selectores



Ceibal



La recomendación [Selectors Level 3 \(2º edición\)](#), publicada en noviembre de 2018, define la sintaxis de los selectores, las [pseudo-clases](#) y los [pseudo-elementos](#).

Selectores de etiqueta

👉 Se usa para **aplicar estilos a todos los elementos de un mismo tipo**, por ejemplo, que todos los párrafos sean azules.

👉 El selector de etiqueta coincide con la etiqueta html del elemento a modificar.

👉 Es un método mucho menos específico para agregar estilos.

1L

```
<p>Esto es un párrafo &lt;p&gt;.</p>
<p>Esto es otro párrafo &lt;p&gt;.</p>
```

NOTA: En html no podemos simplemente usar los caracteres '<>' ya que están reservados para ser usados en las etiquetas, por lo que si queremos usarlos en el texto, debemos codificarlos como en el ejemplo: '<' y '>'.

```
p {
  color: red;
}
```

Esto es un párrafo <p>.

Esto es otro párrafo <p>.

Selector *universal*: *

Si se escribe un *asterisco* (*), las propiedades afectan a todos los elementos de la página. En el ejemplo siguiente, tanto el párrafo <p> como la dirección <address> se ven de color rojo.

```
<p>Esto es un párrafo &lt;p&gt;.</p>
<address>Esto es una dirección &lt;address&gt;.</address>
```

```
* {
  color: red;
}
```

Esto es un párrafo <p>.

Esto es una dirección <address>.

```
/* Aplica a TODOS los elementos */

* {

  box-sizing: border-box; /* Incluye padding y border en el tamaño total del
  elemento */

  margin: 0; /* Elimina los márgenes por defecto de todos los elementos */

  padding: 0; /* Elimina el relleno por defecto de todos los elementos */

}
```

Selector de *clase*: Atributo **Class**

👉 Las clases se utilizan cuando quieres aplicar el mismo estilo a más de un elemento, y la búsqueda por etiqueta no sirve para lograrlo.

👉 No necesitas escribir varias veces el mismo CSS, ni repetir el ID, simplemente utiliza una clase aplicada a todos los elementos que deban compartir el estilo.

Desde CSS, puedes **usar los nombres que quieras**, siempre y cuando empiecen con **letras**, y pongas un "." adelante. Lo recomendable es poner un nombre que haga referencias a los estilos que tendrá. Por ejemplo:

```
/* selector de clase */
```

```
.destacado {
```

```
font-weight: bold;
```

```
}
```

Selector de *clase*: **E.e**

Si se escribe una etiqueta seguida de un punto y un nombre de clase (E.e), **las propiedades afectan a todos los elementos con etiqueta E cuyo atributo class tenga el valor e**. En el ejemplo siguiente, sólo el párrafo <p> de clase "aviso" se ve de color rojo.

```
<p>Esto es un párrafo &lt;p&gt;.</p>  
<p class="aviso">Esto es un párrafo &lt;p&gt; de clase "aviso".</p>
```

```
p.aviso {  
  color: red;  
}
```

Esto es un párrafo <p>.

Esto es un párrafo <p> de clase "aviso".

Para definir una clase que se pueda aplicar a cualquier etiqueta, se puede escribir como selector un punto y el nombre de la clase (.nombre_clase) o el selector universal y el nombre de la clase (*.nombre_clase). En los dos ejemplos siguientes, cualquier elemento de clase "aviso" se ve de color rojo.

```
<p>Esto es un párrafo sin clase <span class="aviso">pero</span> con un &lt;span&gt; de clase "aviso" (en la palabra "pero").</p>
<p class="aviso">Esto es un párrafo &lt;p&gt; de clase "aviso".</p>
<address class="aviso">Esto es una dirección &lt;address&gt; de clase "aviso".</address>
```

```
.aviso {
  color: red;
}
```

Esto es un párrafo sin clase **pero** con un de clase "aviso" (en la palabra "pero").

Esto es un párrafo <p> de clase "aviso".

Esto es una dirección <address> de clase "aviso".

```
*.aviso {
  color: red;
}
```

Esto es un párrafo sin clase **pero** con un de clase "aviso" (en la palabra "pero").

Esto es un párrafo <p> de clase "aviso".

Esto es una dirección <address> de clase "aviso".

Selector de *identificador (id)* : Atributo ID

👉 Generalmente se usa para nombrar porciones de código y sectores, como por ejemplo cuando quieres nombrar distintas secciones.

👉 Es posible ponerle ID a cualquier elemento HTML para darle un "nombre".

👉 No necesitas escribir varias veces el mismo CSS, ni repetir el ID.

Desde CSS, puedes **usar los nombres que quieras**, siempre y cuando empiecen con **letras**, y pongas un "#" adelante. Lo recomendable es poner un nombre que haga referencias a los estilos que tendrá. Por ejemplo:


```
/* selector de
ID*/
#princip
al {
background-color:
yellow;
}
```

Selector de *identificador (id)* : E#e

Si se escribe una etiqueta seguida de una almohadilla y un nombre de id (E#e), las propiedades afectan al elemento con etiqueta E cuyo atributo *id* tenga el valor 'e'. En el ejemplo siguiente, sólo el párrafo <p> de id 'aviso' se ve de color rojo.

```
<p class="aviso">Esto es un párrafo &lt;p&gt; de clase "aviso".</p>
<p id="aviso">Esto es un párrafo &lt;p&gt; con id "aviso".</p>
```

```
p#aviso {
color: red;
}
```

Esto es un párrafo <p> de clase "aviso".
 Esto es un párrafo <p> con id "aviso".

Hay que tener en cuenta que **el atributo id no se puede repetir** en la misma página web, es decir, no puede haber dos elementos con el mismo valor del atributo id (independientemente de que los elementos tengan etiquetas iguales o distintas).

Para definir un id que se pueda aplicar a cualquier etiqueta, se puede escribir como selector una almohadilla y el nombre del id (#nombre-id) o el selector universal y el nombre del id (*#nombre-id).

En los dos ejemplos siguientes, cualquier elemento con id "aviso" se ve de color rojo.

<pre>#aviso { color: red; }</pre>	Esto es un párrafo <p> de clase "aviso". Esto es un párrafo <p> con id "aviso".
<pre>*#aviso { color: red; }</pre>	Esto es una dirección <address> de clase "aviso". Esto es una dirección <address> con id "aviso".



```
<section id= "prod">  
  <article class= "rojo">  
  </article>  
  <article id= "prod">  
  </article>  
</section>
```

```
<section id= "prod">  
  <article class= "rojo">  
  </article>  
  <article class= "rojo">  
  </article>  
</section>
```



Tanto **ID** como **Class** pueden ser utilizadas dentro del HTML en diferentes etiquetas. Sin embargo, **los nombres otorgados a las clases se pueden repetir**, mientras que utilizados en **los IDs no**.

Aunque en una página web no puede haber dos elementos con el mismo valor del atributo id, en páginas distintas sí puede haber dos elementos con el mismo valor del atributo id.

Si la misma hoja de estilo se enlaza desde varias páginas web, el selector universal afectará en cada página a los elementos con el id correspondiente.

Selector de *descendentes* : E F

Si se escriben dos etiquetas separadas por un espacio (E F) las propiedades afectan a los elementos con la segunda etiqueta (F) contenidos dentro de la primera etiqueta (E), aunque haya etiquetas intermedias. En el ejemplo siguiente, todos los párrafos <p> dentro de una división <div> se ven de color rojo.

```

<p>Esto es un párrafo &lt;p&gt;.</p>

<div>
  <p>Esto es un párrafo &lt;p&gt;.</p>

  <ul>
    <li>Esto es una lista
      <p>Esto es un párrafo &lt;p&gt; dentro de una lista</p>
    </li>
  </ul>

  <p>Esto es otro párrafo &lt;p&gt;.</p>

  <address>Esto no es un párrafo &lt;p&gt;, sino una dirección &lt;address&gt;.</address>
</div>

<p>Esto es otro párrafo &lt;p&gt;.</p>

```

```

div p {
  color: red;
}

```

Esto es un párrafo <p>.

Esto es un párrafo <p>.

- Esto es una lista

Esto es un párrafo <p> dentro de una lista

Esto es otro párrafo <p>.

Esto no es un párrafo <p>, sino una dirección <address>.

Esto es otro párrafo <p>.

Si se escriben varias etiquetas seguidas (E F G ...), **las propiedades afectan a los elementos con la última etiqueta, contenidos dentro de la etiqueta anterior**, contenidos a su vez dentro de la etiqueta anterior y así sucesivamente, aunque haya etiquetas intermedias. En el ejemplo siguiente, únicamente los párrafos <p> dentro de un elemento de lista dentro de una división <div> se ven de color rojo.

```
div li p {
  color: red;
}
```

Esto es un párrafo <p>.

Esto es un párrafo <p>.

- Esto es una lista

Esto es un párrafo <p> dentro de una lista

Esto es otro párrafo <p>.

Esto no es un párrafo <p>, sino una dirección <address>.

Esto es otro párrafo <p>.

Selector de *hijos* : E > F

Si se escriben dos etiquetas seguidas separadas por un signo "mayor que" (E > F), las propiedades afectan a los elementos con la segunda etiqueta (F) contenidos dentro de la primera etiqueta (E), pero no lo selecciona si hay etiquetas intermedias entre ellos. En el ejemplo siguiente, únicamente los párrafos <p> dentro de una división <div> sin elementos intermedios, se ven de color rojo. El párrafo que se encuentra dentro de un , no se seleccionará, ya que si bien es *descendiente* de un <div>, no es *hijo* del mismo (es *hijo* del).

```
div > p {
  color: red;
}
```

Esto es un párrafo <p>.

Esto es un párrafo <p>.

- Esto es una lista

Esto es un párrafo <p> dentro de una lista

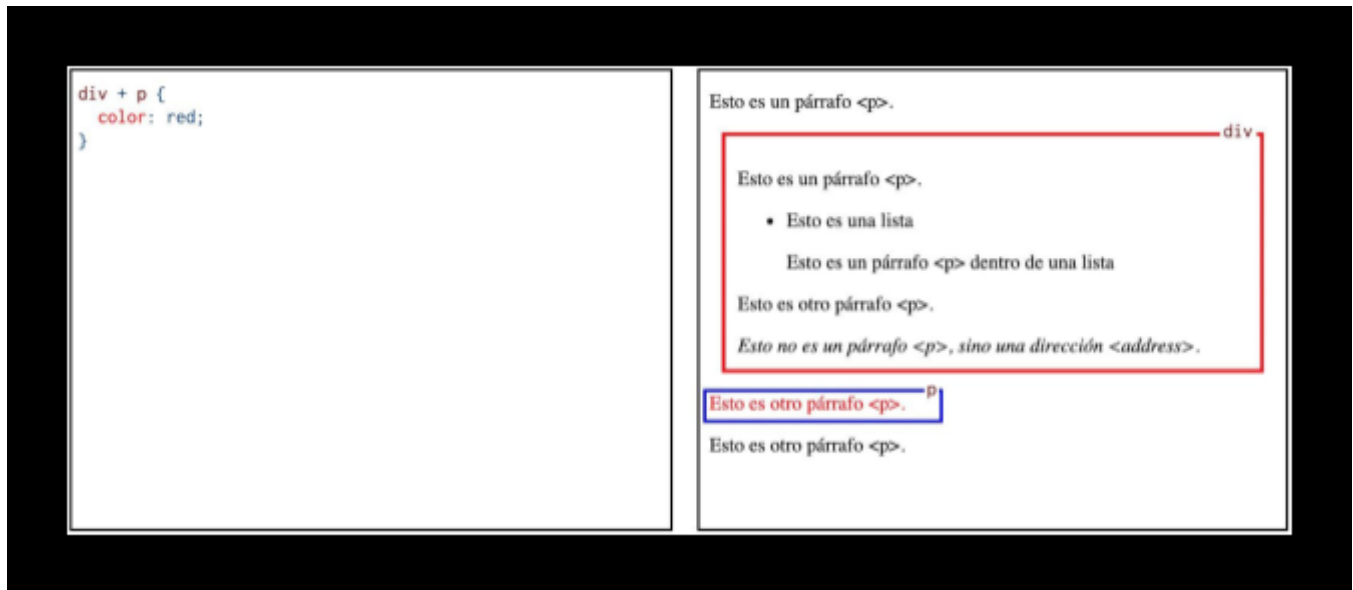
Esto es otro párrafo <p>.

Esto no es un párrafo <p>, sino una dirección <address>.

Esto es otro párrafo <p>.

Selector de *consecutivos* : E + F

Si se escriben etiquetas separadas por un signo "más" (E + F), las propiedades afectan únicamente a los elementos con la última etiqueta (F) que van justo después de un elemento con la etiqueta (E). En el ejemplo siguiente, únicamente el párrafo <p> inmediatamente posterior a la división <div> se ve de color rojo.



Para los selectores de **hijos**, **descendientes** y **consecutivos**, podemos añadir mucho más de dos elementos. Hay que tener en cuenta que el estilo se aplicará siempre al último elemento (el de más a la derecha).

En el **selector de hijos**, si tenemos: $A > B > C > D$ el estilo se aplicará solamente a los elementos D , que sean 'hijos' de un elemento C , que a su vez sea 'hijo' de un elemento B , que a su vez sea 'hijo' de un elemento A .

En el **selector de descendientes**, si tenemos: $A B C D$ el estilo se aplicará solamente a los elementos D , que sean 'descendientes' de un elemento C , que a su vez sea 'descendiente' de un elemento B , que a su vez sea 'descendiente' de un elemento A .

En el **selector de consecutivos**, si tenemos: $A + B + C + D$ el estilo se aplicará solamente a los elementos D , que se encuentren exactamente después de un elemento C , que a su vez se encuentre exactamente después de un elemento B , que a su vez se encuentre exactamente después de un elemento A .

Selector de grupo : E, F

Los selectores de grupo, son en realidad varios selectores 'unidos' para darles el mismo estilo a todos los elementos seleccionados. En este caso, no se dará estilo solamente al último elemento, sino a todos.

```
p, address {  
  color: red;  
}
```

Esto es un párrafo <p>.

Esto es una dirección <address>.



Todos los selectores se pueden además combinar entre sí para lograr seleccionar exactamente los elementos deseados.

¿Qué elementos pintará el siguiente selector?

```
h3, div#central+div p>span.muestra {  
  color: red;  
}
```

Pintará de rojo todos los títulos de tercer nivel <h3>, así como todos los contenedores en línea de la clase "muestra", que sean *hijos* de un párrafo <p>, que a su vez se encuentre dentro (sea *descendiente*) de un contenedor en bloque <div>, que a su vez esté ubicado exactamente después de un contenedor de bloque <div> con el identificador "central"...

Sí, suena algo entreverado, por eso la mejor manera de entenderlo es **practicando**

Precedencia de declaraciones

Cuando reglas distintas apuntan al mismo objeto:

- ✓ Si son **propiedades distintas**, se suman (se combinan).
- ✓ Si tienen **alguna propiedad repetida**, solo una queda.

Si las propiedades entran en conflicto, la regla que se aplica es la del selector de mayor **especificidad**.

La **especificidad** de un selector se puede calcular atendiendo a los siguientes criterios o **precedencia**:

- ✓ **ID** pisa cualquier otra regla.
- ✓ **Class** sobrescribe las reglas de etiqueta, pero no las de **ID**.
- ✓ **Etiquetas** tienen la menor precedencia.

Cuando múltiples orígenes CSS definen la misma propiedad para un elemento, la precedencia es:

1. **Estilos en línea, atributo `style=".."`** (mayor prioridad).
2. **Estilos en etiqueta `<style>`**.
3. **Estilos en hojas de estilo (.css) enlazadas**.
4. **Estilos por defecto del navegador** (menor prioridad).



👉 La declaración `!important`; corta la precedencia.

Se escribe después del valor de la propiedad CSS que se quiere convertir en la más importante. Se utiliza un `!important`; por cada valor a pisar.

Si necesitas más de 5 `!important`; en todo tu CSS, algo estás haciendo mal 😞.



A practicar:

CSS Diner es un juego interactivo diseñado para enseñarte selectores de CSS. En cada nivel, se te presentará una mesa con diferentes elementos visuales (como platos, bentos o frutas) y una instrucción que te indicará exactamente cuáles de esos elementos debes seleccionar.

Tu tarea es escribir el selector de CSS correcto en el editor proporcionado debajo de la mesa; si tu selector coincide precisamente con los elementos solicitados, estos se resaltarán, confirmando tu éxito y permitiéndote avanzar al siguiente nivel con un nuevo desafío de selectores.

CSS Editor

style.css

HTML Viewer

table.html

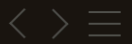
```
1 Type in a CSS selector
2 {
3 /* Styles would go here. */
4 }
5
6 /*
7 Type a number to skip to a level.
8 Ex → "5" for level 5
9 */
10
11
12
13
14
15
16
17
18
19
20
```

enter

```
1 <div class="table">
2   <plate />
3   <plate />
4 </div>
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Made by @flukeout — come say hi!

Level 1 of 32 ✓



Type Selector

Select elements by their type

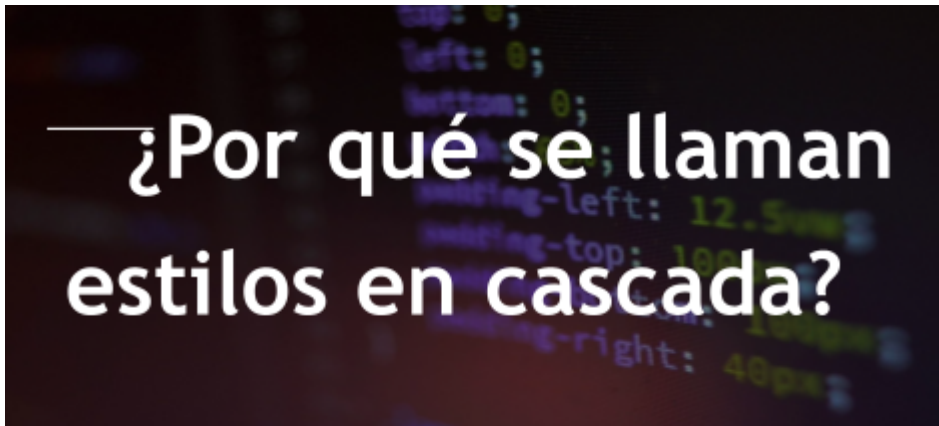
A

Selects all elements of type `A`. Type refers to the type of tag, so `<div>`, `<p>` and `<ul>` are all different element types.

Examples

`div` selects all `<div>` elements.`p` selects all `<p>` elements.

Estilos en Cascada y Herencia



Las hojas de estilo se llaman hojas de estilo **"en cascada"** porque:

Las propiedades de estilo pueden estar escritas en varios sitios

(en varios lugares de la página web o de la hoja de estilo) y dependiendo del sitio, afectan a más o menos elementos

Cuando un elemento está *contenido* en otro (por ejemplo, un párrafo <p> dentro de una división <div>), **al elemento de dentro se le aplican también las propiedades definidas para el elemento de fuera** (al párrafo <p> se le aplicarían las propiedades definidas para la división <div>).

Dos reglas distintas pueden ser de aplicación a un mismo elemento (por ejemplo, a un párrafo <p> con *clase* "nombre", le es de aplicación tanto el selector **.nombre** como el selector **p**).

Si las propiedades (escritas en diferentes sitios o para diferentes elementos) no entran en conflicto, el navegador **aplica todas las propiedades**.

Por ejemplo, si el color de fondo de un elemento está definido en un sitio y el tamaño de letra en otro sitio, el navegador aplicará ambas propiedades al elemento.

Si las propiedades en conflicto se encuentran definidas en diferentes hojas de estilo, el navegador aplica el valor definido en la última hoja de estilo cargada (es decir, en la última).

Elementos dentro de otros.

Cuando un elemento está contenido en otro (por ejemplo, un párrafo `<p>` dentro de una división `<div>`), al elemento de dentro se le aplican también las propiedades definidas para el elemento de fuera. Cuando esto ocurre decimos que el elemento **hereda** dicha propiedad.

En el ejemplo siguiente, el párrafo incluido dentro de la división se muestra de color rojo.

```
<div>
  <p>Esto es un párrafo dentro de una división.</p>
</div>
```

```
div {
  border: black 3px solid; color: red;
}
```

Esto es un párrafo dentro de una división.

Si una misma propiedad está definida para el elemento inferior y para el superior, se aplica el valor establecido para el elemento inferior.

la propiedad definida específicamente para un elemento, tendrá prioridad por sobre la propiedad heredada

```
div {
  border: black 3px solid;
  color: red;
}

p {
  color: black;
}
```

Esto es un párrafo dentro de una división.

```
p {
  color: black;
}

div {
  border: black 3px solid;
  color: red;
}
```

Esto es un párrafo dentro de una división.

Para pensar: cascada
El navegador lee de arriba
hacia abajo (forma de
cascada)

¿De qué color crees que se
aplicará al párrafo (p) al ver
el siguiente código?

```
p {  
  color: red;  
}  
  
p {  
  color: green;  
}
```

El Concepto de Padre e Hijo en HTML

Imagina tu documento HTML como un árbol genealógico (o más precisamente, una estructura de árbol). En esta estructura, los elementos HTML pueden contener a otros elementos.

- Un elemento **padre** es un elemento que **contiene** a otro u otros elementos dentro de sus etiquetas de apertura y cierre.
- Un elemento **hijo** es un elemento que está **contenido** dentro de otro elemento (su padre).

Por ejemplo:

```
<div class="contenedor-padre">
```

```
  <p class="elemento-hijo-1">Este es un párrafo dentro del div.</p>
```

```
  <span class="elemento-hijo-2">Este es un span dentro del mismo div.</span>
```

```
</div>
```

- **En este ejemplo:**

- El elemento `<div>` es el **padre** de los elementos `<p>` y `<span>`.
- Los elementos `<p>` y `<span>` son **hijos** del `<div>`.

A su vez, si el `<div>` estuviera dentro de otro elemento, el `<div>` sería hijo de ese elemento superior.

- Esta relación jerárquica es inherente a la estructura de un documento HTML. Los elementos se anidan unos dentro de otros para organizar el contenido.

¿Por qué es importante esta relación?

La relación padre-hijo es vital porque los estilos CSS y el comportamiento de maquetación de un elemento **a menudo dependen de su contenedor padre**.

Propiedades como el ancho (`width`), la altura (`height`), el porcentaje de tamaño, y algunas formas de posicionamiento (profundizaremos mas adelante sobre esto), se calculan o se aplican **en relación con las dimensiones o el contexto de su padre**.

	
https://www.youtube.com/watch?v=aCGjEUWyjhq	

Tamaños y distancias

Tamaño: width y height

La propiedad **width** establece el ancho de un elemento mediante un valor numérico (con su unidad) o un porcentaje (con respecto al elemento padre).

Si no se establece la anchura de un elemento, el elemento ocupa todo el ancho disponible.

```
p {  
  border: black 1px solid;  
}
```

¡Hola, mundo!

```
p {  
  border: black 1px solid;  
  width: 50%;  
}
```

¡Hola, mundo!

```
p {  
  border: black 1px solid;  
  width: 20em;  
}
```

¡Hola, mundo!

```
p {  
  border: black 1px solid;  
  width: 200px;  
}
```

¡Hola, mundo!

La propiedad **height** establece la altura de un elemento mediante un valor numérico (con su unidad), pero no en porcentaje. Si no se establece la altura de un elemento, el elemento ocupa el espacio necesario para mostrar su contenido (sin contar los elementos flotantes).

<pre>p { border: black 1px solid; }</pre>	Si no se establece la altura de un elemento, el elemento ocupa el espacio necesario para mostrar su contenido (sin contar los elementos flotantes).
<pre>p { border: black 1px solid; height: 50%; /* Los navegadores no hacen caso de esta propiedad */ }</pre>	¡Hola, mundo!
<pre>p { border: black 1px solid; height: 3em; }</pre>	¡Hola, mundo!
<pre>p { border: black 1px solid; height: 100px; }</pre>	¡Hola, mundo!

Si se establece la **anchura** y **altura** de un elemento, el contenido del elemento puede no caber en el tamaño asignado. La propiedad **overflow** establece el comportamiento en estos casos.

Tamaño de imágenes

Las propiedades **width** y **height** también se pueden aplicar a imágenes (esos valores tienen prioridad sobre los establecidos en los **atributos width y height** de la etiqueta **** respectivamente).

- si sólo se indica una propiedad (width o height), la imagen cambia de tamaño manteniendo la proporción.
- si se indican las dos propiedades (width y height), la imagen cambia al tamaño indicado.

Distancias y tamaños

En una página web o en una hoja de estilo se pueden definir las distancias o tamaños como porcentajes o como valores numéricos (absolutos o relativos) teniendo en cuenta los siguientes criterios:

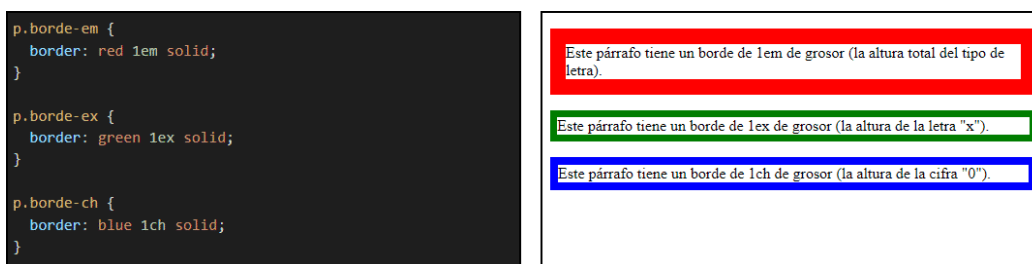
- los porcentajes se escriben con el símbolo de porcentaje pegado al valor numérico (sin espacio entre ellos). Por ejemplo, 100%, 50%, etc.
- los valores numéricos pueden ser positivos o negativos, enteros o decimales (utilizando el punto como separador de las partes entera y decimal). Las unidades deben ir pegadas a los valores numéricos (sin espacio entre ellos). Por ejemplo, 10px, 0.5cm, -10pt, etc.
- cuando el valor es cero, se aconseja no escribir la unidad (por ejemplo, escribir 0 en vez de 0px)

Unidades **relativas**

Dentro de las unidades relativas podemos identificar dos tipos: las unidades relativas definidas **respecto al tipo de letra** y las unidades relativas definidas **respecto a la pantalla**.

Unidades relativas respecto al tipo de letra:

- **em**: el tamaño de fuente del elemento contenedor (se puede relacionar directamente con los porcentajes, concretamente 1em = 100%.)
- **ex**: la altura de la letra "x" de la fuente del elemento contenedor
- **ch**: la altura de la cifra 0 (cero) de la fuente del elemento contenedor
- **rem**: el tamaño de la fuente del elemento raíz de la página En la práctica, las más utilizadas son **em** y **rem**.



En estas unidades, el tamaño depende del tipo de letra que se esté utilizando. En los ejemplos siguientes, los párrafos tienen un borde con un grosor 1em. Como cada fuente tiene una altura total diferente, el grosor del borde es distinto en cada caso.

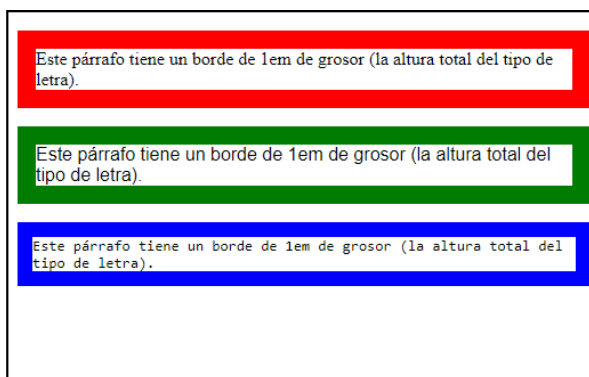

```

p.borde-em-serif {
  border: red 1em solid;
  font-family: serif;
}

p.borde-em-sans-serif {
  border: green 1em solid;
  font-family: sans-serif;
}

p.borde-em-monospace {
  border: blue 1em solid;
  font-family: monospace;
}

```



Unidades relativas respecto a la pantalla:

- **vw**: el 1% del ancho de la ventana (sin contar bordes, pestañas barras de desplazamiento, etc.)
- **vh**: el 1% del alto de ventana (sin contar bordes, pestañas barras de desplazamiento, etc.)
- **vmin**: el menor de los tamaños **vw** y **vh**
- **vmax**: el mayor de los tamaños **vw** y **vh**

Unidades absolutas

- **px**: píxeles, la 96ava parte de una pulgada (96 px = 1in = 2.54cm)
- **cm**: centímetros
- **mm**: milímetros
- **Q**: cuartos de milímetros
- **in**: pulgadas (1in = 2.54cm = 25.4mm)
- **pc**: picas, la sexta parte de una pulgada (6pc = 1in)
- **pt**: puntos, la doceava parte de una pica, es decir, la 72ava parte de una pulgada (1in = 6pc = 72pt)

En la práctica, las más utilizadas son **px** y **pt**.

Porcentajes %

Cuando se emplean porcentajes para expresar el tamaño del tipo de letra, los valores superiores a 100% significan una ampliación y los valores inferiores a 100% significan reducción.

```
p {  
  font-size: 200%;  
}
```

Este párrafo está aumentado al 200%
(el doble del tamaño normal).

```
p {  
  font-size: 75%;  
}
```

Este párrafo está reducido al 75% del tamaño normal.

Cuando se emplean porcentajes para expresar distancias o tamaños de elementos, los porcentajes se interpretan con respecto al elemento contenedor (cuando se refieren a distancias horizontales), o respecto al elemento contenedor o el tamaño de la ventana (cuando se refieren a distancias verticales).

```
hr {  
  width: 50%;  
}
```

Este separador ocupa el 50% del espacio horizontal disponible:

Debido a que las propiedades se aplican en cascada, al cambiar el tamaño de un elemento, los elementos contenidos en él también modifican su tamaño. En el ejemplo siguiente, el párrafo situado en la división `<div>` aumenta su tamaño al 150%

```
<p>Párrafo fuera de una división.</p>  
  
<div>  
  <p>Párrafo dentro de una división.</p>  
</div>
```

```
div {  
  border: black 2px solid;  
  font-size: 150%;  
}
```

Párrafo fuera de una división.

Párrafo dentro de una división.



Si se cambia el tamaño en dos elementos anidados, los porcentajes se multiplican entre sí. A este fenómeno se le denomina "*composición*", es decir, el tamaño final es la composición de tamaños de cada elemento.

En el ejemplo siguiente, los párrafos están aumentados al 150%, por lo que el párrafo situado en la división (cuyo contenido está a su vez aumentado al 150%), está aumentado el 225% (es decir, toma como referencia el tamaño de letra del contenedor, por lo que aumenta al 150% del que ya está aumentado al 150%).

```
<p>Este párrafo está aumentado al 150%.</p>

<div>
  <p>Este párrafo está aumentado al 150%.</p>
</div>

div {
  border: black 1px solid;
  font-size: 150%;
}

p {
  font-size: 150%;
}
```

Este párrafo está aumentado al 150%.

Este párrafo está aumentado al 150%.

Esta composición de tamaños no siempre es conveniente y puede provocar cambios de tamaños indeseados debido a anidamientos no previstos. Para evitarlo, se puede utilizar la unidad relativa **rem**, que toma como referencia el tamaño de la fuente del elemento raíz de la página .

Entendiendo las Unidades de Medida en CSS

En CSS, usamos unidades para decirle al navegador qué tan grande o pequeña debe ser una propiedad (como el ancho, la altura, el tamaño del texto, etc.). Pero no todas las unidades miden lo mismo de la misma manera. ¡Aquí te mostramos las diferencias clave!

Imagina que cada **píxel (px)** en tu pantalla es como un diminuto cuadradito de color. Algunas unidades miden contando estos cuadraditos directamente (unidades absolutas), y otras miden en relación a otra cosa (unidades relativas).

px (Píxel) - El Cuadradito Fijo

El **píxel (px)** es la unidad más sencilla. Mide un número fijo de esos cuadraditos en tu pantalla. Es una **unidad absoluta**: 150px siempre serán 150 cuadraditos, sin importar nada más.

Es útil cuando necesitas un tamaño muy preciso que no debe cambiar.

Representación simple:

```
[■■■■■■■■■■] → Esto es 100px de ancho (representación simple de 100 cuadraditos).
[■■■■■■■■■■■■■■■■■■■■] → Esto es 150px de ancho.
```

Código HTML del ejemplo:

```
<div class="caja-ejemplo">
  width: 150px;
</div>
```

Código CSS específico del ejemplo:

```
.ejemplo-px .caja-ejemplo {
  width: 150px; /* Ancho fijo en píxeles */
  height: 50px;
  background-color: #dc3545; /* Rojo */
  line-height: 50px;
}
```

width: 150px;

Esta caja mide exactamente **150 cuadraditos** de ancho. Su tamaño NO cambia si agrandas el texto o la ventana.

Consigna: Abre las herramientas de desarrollador (F12). Selecciona la caja roja. En la pestaña "Estilos", busca la regla ``width: 150px``. Haz doble clic en ``150px`` y prueba a cambiar el número (ej. ``200px``, ``50px``). Observa cómo cambia el tamaño fijo de la caja.

em - Relativo al Padre (Fuente)

La unidad **em** mide en relación al **tamaño de la fuente del elemento padre**. Si el padre tiene una letra de 18px (18 cuadraditos) y el hijo tiene font-size: 1.5em;, el hijo tendrá una letra de 27px (18 * 1.5 = 27).

Su tamaño depende directamente de la letra de su papá. 1em es igual al tamaño de fuente del padre.

Representación simple:

```
Padre (font-size: 18px): [█] (1 * 18px)
```

```
Hijo (font-size: 1.5em): [██████████████████] (1.5 * 18px)
```

Código HTML del ejemplo:

```
<div class="padre">
  Soy el padre. Mi letra es de 18px.
  <div class="hijo-em caja-ejemplo">
    Soy el hijo. Mi letra es 1.5em.
  </div>
</div>
```

Código CSS específico del ejemplo:

```
.ejemplo-em .padre {
  font-size: 18px; /* Tamaño de fuente del padre */
  background-color: #28a745; /* Verde */
  padding: 15px;
  color: #fff;
}

.ejemplo-em .hijo-em {
  font-size: 1.5em; /* 1.5 veces el tamaño del padre */
  background-color: #ffc107; /* Amarillo */
  color: #333;
  padding: 10px;
  margin-top: 10px;
}
```

Soy el padre. Mi letra es de **18px**. Soy el hijo. Mi letra es **1.5em** (¡basada en la letra de mi padre!).

Si cambias el tamaño de fuente del padre, el hijo con em cambiará también.

Si cambias el tamaño de fuente del padre, el hijo con em cambiará también.

Consigna: Abre las herramientas de desarrollador (F12). Selecciona el DIV padre (verde). En la pestaña "Estilos", busca la regla `font-size: 18px;`. Cambia `18px` por otro valor (ej. `14px`, `24px`). Observa cómo el tamaño del texto (y la caja) del hijo con `em` cambia automáticamente.

rem - Relativo a la Raíz (Fuente)

La unidad **rem** (root em) mide en relación al **tamaño de la fuente del elemento raíz del documento** (`<html>`). Por defecto, 1rem suele ser 16px (16 cuadraditos).

Si usas `font-size: 1.5rem;`, tu letra será 1.5 veces el tamaño de la letra raíz (ej. $1.5 * 16\text{px} = 24\text{px}$), SIN importar el tamaño de letra de tus padres directos.

Es genial para tener una escala de texto consistente en toda tu página que responda a las preferencias de tamaño de texto del usuario.

Representación simple:

HTML Raíz (font-size: 16px por defecto): [■■■■■■■■■■■■■■■■■■■■]

Elemento (font-size: 1.5rem): [■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■] (1.5 * 16px)

Código HTML del ejemplo:

```
<div class="padre-diferente">
  Soy un padre con letra de 20px.
  <div class="hijo-rem caja-ejemplo">
    Soy un hijo. Mi letra es 1.5rem.
  </div>
</div>
```

Código CSS específico del ejemplo:

```
html {
    font-size: 16px; /* Tamaño de fuente raíz */
}

.ejemplo-rem .padre-diferente {
    font-size: 20px; /* Padre con fuente diferente */

    background-color: #6f42c1; /* Morado */
    padding: 15px;
    color: #fff;
}

.ejemplo-rem .hijo-rem {
    font-size: 1.5rem; /* 1.5 veces el tamaño de la raíz */
    background-color: #20c997; /* Verde azulado */
    color: #333;
    padding: 10px;
}
```

Soy un padre con letra de 20px. Soy un hijo. Mi letra es **1.5rem** (¡basada en la letra de la RAÍZ!).

El tamaño de esta caja rem solo depende del tamaño de fuente que definas para la etiqueta `<html>` (generalmente 16px).

Consigna: Abre las herramientas de desarrollador (F12). Selecciona la etiqueta ``. En la pestaña "Estilos", busca la regla `'html { font-size: 16px; }'` (o la regla por defecto). Cambia `16px` por otro valor (ej. `10px`, `20px`). Observa cómo el tamaño del texto (y la caja) con `rem` cambia, mientras que el texto del padre (20px) no lo hace.

% (Porcentaje) - Relativo al Padre (Tamaño)

El **porcentaje (%)** mide en relación al **tamaño del elemento padre**. Si el padre tiene un ancho de 400px (400 cuadraditos) y el hijo tiene `width: 50%;`, el hijo tendrá un ancho de 200px (200 cuadraditos, que es el 50% de 400).

Su tamaño es una fracción del tamaño de su padre.

Representación simple:

Código HTML del ejemplo:

```
<div class="padre-ancha">
  Padre (ancho: 400px)
  <div class="hijo-porcentaje caja-ejemplo">
    Hijo (width: 50%)
  </div>
</div>
```

Código CSS específico del ejemplo:

Es una unidad **relativa a la ventana**, ideal para elementos que deben ocupar una fracción de la altura total de la pantalla, como una sección que siempre ocupa la pantalla completa al cargar (height: 100vh;).

Representación simple:

```
Ventana (height: 800px):
[██████████] (Parte superior)
[██████████]
[██████████] (Elemento con height: 40vh, aprox 320px si la ventana es 800px)
[██████████]
[██████████]
[██████████]
[██████████] (Parte inferior)
```

Código HTML del ejemplo:

```
<div class="caja-ejemplo">
  height: 40vh;
</div>
```

Código CSS específico del ejemplo:

```
.ejemplo-vh .caja-ejemplo {
  height: 40vh; /* 40% de la altura de la ventana */
  background-color: #fd7e14; /* Naranja */
  width: 200px; /* Ancho fijo para demostrar la altura variable */
  /* Centrar contenido */
  display: flex;
  justify-content: center;
  align-items: center;
}
```

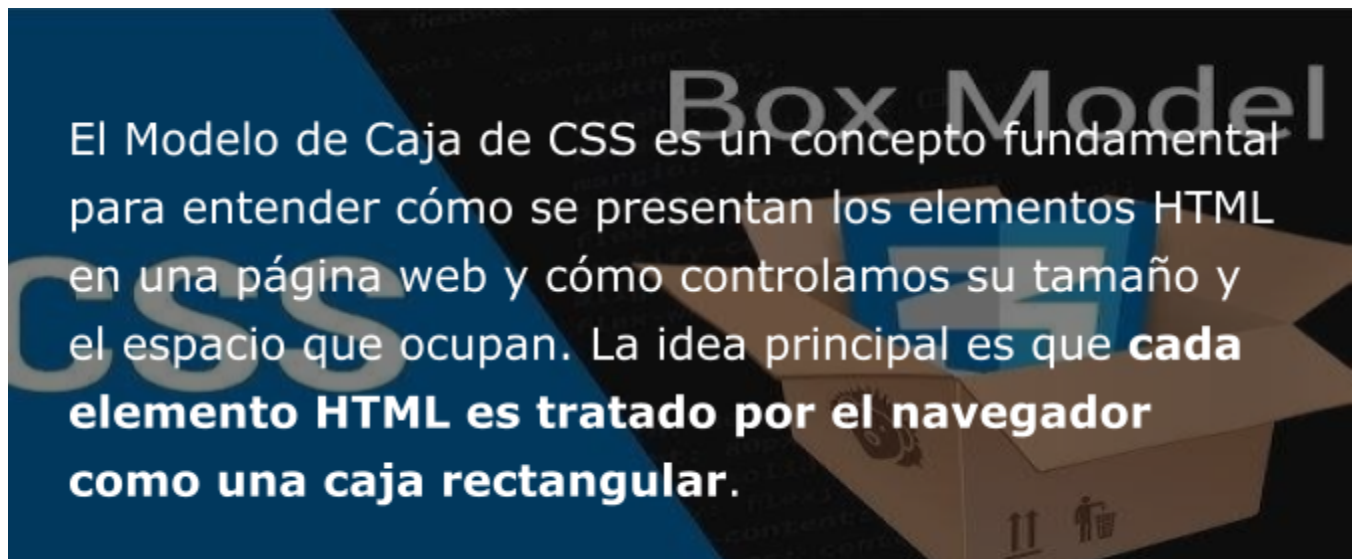
height: 40vh;

La altura de esta caja es el 40% de la **altura de tu ventana**. ¡Prueba a redimensionar la ventana del navegador para verlo cambiar!

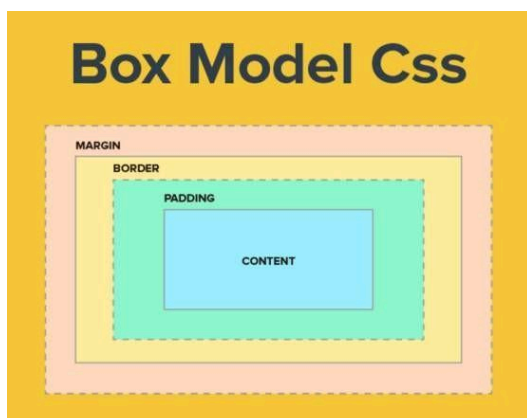
Consigna: Haz la ventana de tu navegador más alta y luego más baja. Observa cómo la altura de la caja cambia dinámicamente para mantener siempre el 40% de la altura actual de la ventana. Intenta usar la barra de scroll si es necesario.

Modelo de caja (Box Model)

Ceibal



Esta caja está compuesta por varias capas, como si fuera una cebolla o una serie de marcos alrededor del contenido. Comprender estas capas es esencial para la maquetación:



CONTENT: el espacio para el texto o imagen.

PADDING: es el espacio libre o “aire” que se deja entre el contenido y el borde de la caja. Es parte del espacio interno del elemento.

BORDER: el límite entre el elemento y el espacio externo.

MARGIN: separación entre el borde y el afuera de la caja. Es parte del espacio externo.

- **Content (Contenido):** Esta es la capa más interna. Es el espacio donde se encuentra el contenido real del elemento, ya sea texto, una imagen, un video u otros elementos HTML anidados. Su tamaño se define principalmente con las

propiedades **width** y **height**.

- **Padding (Relleno):** Esta capa rodea el contenido. Es un espacio **interno** transparente entre el contenido y el borde del elemento. Piensa en él como el "aire" o "colchón" dentro de la caja.

Se define con las propiedades padding-top, padding-right, padding-bottom, padding-left, o la forma abreviada padding. El padding añade espacio *dentro* **del elemento**, empujando el borde hacia afuera desde el contenido.

- **Border (Borde):** Esta capa es el límite visible de la caja, justo entre el padding y el margen. Es la línea que dibuja el contorno del elemento.

Se define con propiedades como border-style (tipo de línea: sólido, punteado, discontinuo, etc.), border-width (grosor) y border-color (color). Al igual que el padding y el margen, puedes definir cada lado por separado (border-top, border-right, etc.) o usar la forma abreviada border. **El borde suma grosor a la caja.**

- **Margin (Margen):** Esta es la capa más externa, que rodea el borde. Es un espacio **externo** transparente entre el borde de esta caja y los bordes de los elementos vecinos. Es la separación entre elementos. Se define con margin-top, margin-right, margin-bottom, margin-left, o la forma abreviada margin.
- **El margen crea espacio *fuera* del elemento, empujando a otros elementos.**

¿Por qué es importante el Modelo de Caja?

Es fundamental porque el **tamaño total que un elemento ocupa en la página** no es solo su width y height. Por defecto, el tamaño total es la suma del **Content + Padding + Border + Margin**.

Por ejemplo, si tienes una caja con width: 200px, padding: 20px, border: 5px solid black, y margin: 10px, el espacio total que ocupará en el layout será:

Ancho total: 10px (margin-left) + 5px (border-left) + 20px (padding-left) + 200px (width) + 20px (padding-right) + 5px (border-right) + 10px (margin-right) = **270px**
Alto total: 10px (margin-top) + 5px (border-top) + 20px (padding-top) + (altura del

contenido) + 20px (padding-bottom) + 5px (border-bottom) + 10px (margin-bottom) =
(Altura del contenido + 70px)

	
https://www.youtube.com/watch?v=tHBzx-mmIUQ	

La Importancia Práctica de Entender el Modelo de Caja

La relevancia práctica de dominar el Modelo de Caja es inmensa y afecta directamente tu capacidad para crear diseños web efectivos:

Control Preciso del Tamaño Total:

El error más común al principio es pensar que el width y height definen el tamaño total que ocupa un elemento. ¡Falso!

Por defecto (con box-sizing: content-box), el tamaño total es la suma del **Content + Padding + Border + Margin**. Entender esto te permite calcular y predecir exactamente cuánto espacio ocupará tu caja en el layout, evitando sorpresas y superposiciones.

Maquetación y Alineación:

El margen (margin) es fundamental para separar elementos y crear el espacio entre ellos.

El padding (padding) es vital para el espacio interno y la legibilidad del contenido dentro de un elemento.

Saber cómo usar ambos te permite controlar la disposición de los elementos en la página y alinearlos correctamente.

Prevención de Problemas de Layout:

Muchos problemas comunes de diseño (elementos que se desbordan, diseños que se rompen) están relacionados con un mal entendimiento del Modelo de Caja y cómo las dimensiones y el espaciado interactúan.

Por ejemplo, si defines un width fijo y luego añades mucho padding y border, la caja se hará más grande de lo que esperabas, pudiendo desbordar su contenedor o empujar a otros elementos de forma no deseada.

Uso Efectivo de box-sizing:

Aunque la presentación se centra en el modelo estándar (content-box), la propiedad box-sizing: border-box; es muy utilizada hoy en día. Con border-box, el width y height que defines incluyen el padding y el border.

Esto simplifica mucho el cálculo del tamaño total y facilita la creación de layouts responsivos, ya que el tamaño definido es el tamaño *real* de la caja (hasta el borde). Entender el modelo estándar te ayuda a apreciar por qué border-box es tan útil.

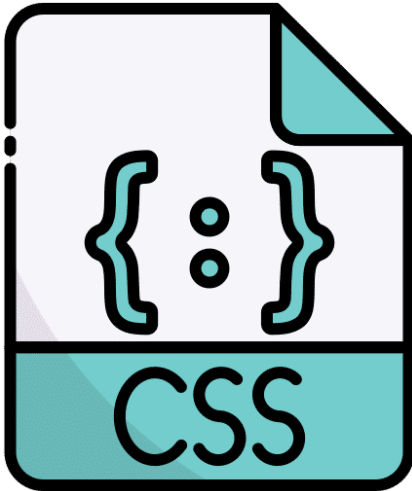
Manejo de overflow:

Cuando una caja tiene dimensiones fijas (width, height) y su contenido es demasiado grande, la propiedad overflow (visible, hidden, scroll, auto) controla cómo se maneja ese contenido extra. Esto es una consecuencia directa de tener un área de contenido definida dentro del Modelo de Caja.

Resumen

Ceibal

Ceibal



La Base es Clave en CSS

Hasta el momento vimos una introducción al mundo de las hojas de estilo, esa herramienta esencial que permite que nuestras páginas web (creadas con HTML) dejen de ser solo estructura y contenido para tener una apariencia visual atractiva y funcional.

No se trata solo de una lista de propiedades, sino de un lenguaje con sus propias reglas y lógica interna.

Piénsate como un arquitecto o diseñador de interiores. No puedes simplemente elegir muebles y colores al azar; necesitas entender los planos de la casa (**HTML**), cómo se construyen las paredes (**el modelo de caja**), cómo fluyen los espacios (**el flujo del documento, controlado por display**), cómo la luz afecta los colores (**la cascada y especificidad**), y cómo los elementos se relacionan entre sí (**padre e hijo, selectores**).

Los conceptos clave que el material presenta, y que son la base para cualquier estilo que apliquemos, aquí te dejamos un resumen:

1. **La Sintaxis Básica:** Entender la estructura selector **{ propiedad: valor; }** es el abecedario de CSS. Sin saber cómo escribir una regla, no podemos aplicar nada.
2. **Selectores:** Son la forma en que "apuntamos" a los elementos HTML que queremos estilizar. Saber usar selectores de tipo, clase, ID, descendientes, etc., nos da el poder de aplicar estilos de forma precisa y controlada.

3. **La Cascada y la Especificidad:** Este es uno de los conceptos más importantes. CSS es "en cascada" porque los estilos pueden venir de diferentes lugares y aplicarse a un mismo elemento. La especificidad es el sistema de reglas que decide qué estilo "gana" en caso de conflicto. Entender esto evita frustraciones cuando un estilo no se aplica como esperas.
4. **Herencia:** Algunas propiedades se transmiten automáticamente de un elemento padre a sus hijos. Conocer qué propiedades se heredan te permite definir estilos generales en elementos contenedores y simplificar tu código.
5. **El Modelo de Caja y las Unidades:** Cada elemento HTML es una caja con contenido, relleno, borde y margen. Comprender este modelo y cómo las diferentes unidades de medida (px, em, rem, %, vw, vh) definen el tamaño de estas partes es crucial para controlar el espacio y la disposición de los elementos.



En resumen, la clave para dominar CSS no es solo saber muchas propiedades, sino entender sus bases: cómo escribir reglas (sintaxis), a quién aplicarlas (selectores), cómo se combinan y priorizan (cascada y especificidad), y cómo se miden los espacios (modelo de caja y unidades).

No dudes en revisar los materiales anteriores tantas veces como sea necesario.

¡Construir sobre una base sólida es esencial!

background-color



La propiedad background-color es fundamental para dar estilo visual a los elementos HTML.

Permite definir el color que cubrirá el área de contenido, el relleno (padding) y el borde (border) de un elemento. Esencialmente, pinta el "fondo" de la caja que representa el elemento en la página.

Puedes especificar el color de diversas maneras:

Usando nombres de colores predefinidos (como blue, red, gray),
Valores hexadecimales (#RRGGBB o #RGB) que son muy precisos
Valores RGB (rgb(rojo, verde, azul)) que especifican la intensidad de cada canal de color (0-255).

Un fondo adecuado mejora la legibilidad del texto y la apariencia general del diseño.

SINTAXIS

```
background-color: blue;
```

Fondo con nombres de colores:

```
body{  
  
background-color: blue;  
  
}
```

Color de fondo en RGB:

```
body{  
  
background-color: rgb(100, 70, 200);  
  
}
```

Color de fondo en hexadecimal:

```
body{  
  
background-color: #92a8d1;  
  
}
```

trinket

Autorun

Share

Save

index.html

styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Background-Color</title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10   <div class="caja-con-fondo">
11     <p>
12       Este párrafo está dentro de una caja con un color de fondo.
13     </p>
14   </div>
15   <div class="caja-con-fondoEx">
16
17     <h2> Explicación del Ejemplo:</h2>
18     <p>Aquí, seleccionamos el div con la clase caja-con-fondo y le
19       aplicamos
20     </p>
21     <h3>background-color:#e0f7fa;</h3>
22
23
24 <p>Esto hace que todo el espacio ocupado por el div, incluyendo el relleno de
25   20px, se pinte de ese color azul verdoso claro. </p>
26
27 <p>El borde también se dibuja sobre este color de fondo. </p>
28
29 <p> El párrafo dentro hereda propiedades o tiene las suyas propias (como el
30   color de texto), pero el fondo es una propiedad del div contenedor.</p>
```

Result

Instructions

Prueba tú mismo:

- Cambia el valor de background-color usando un nombre de color (ej: lightgray) o un valor RGB (ej: rgb(255, 240, 245)). ¿Cómo cambia la apariencia?
- Añade otro div debajo del primero y dale un color de fondo diferente. Observa cómo cada elemento tiene su propio fondo.
- Elimina el padding del .caja-con-fondo. ¿Qué le ocurre al texto en relación con el borde y el fondo?

CONSEJOS

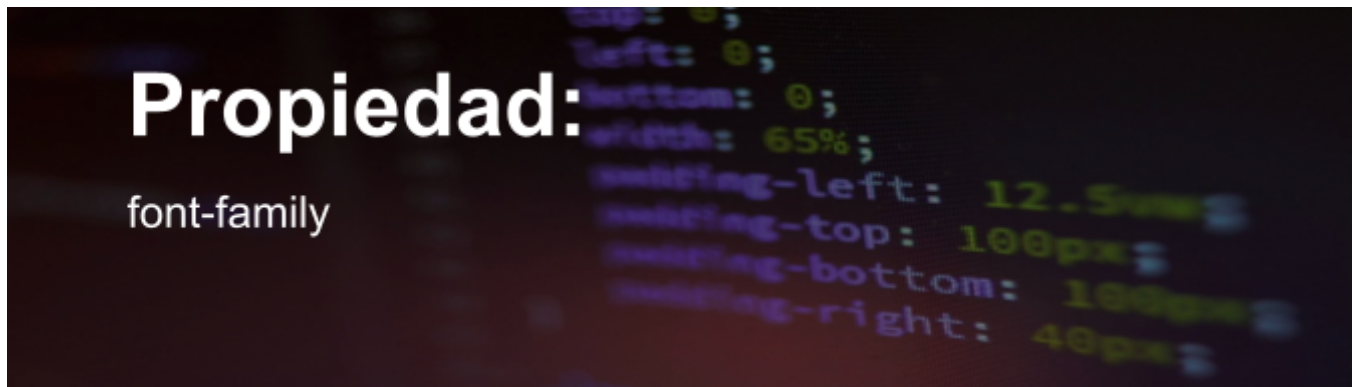


- El color de fondo predeterminado es transparente.
- El fondo de un elemento incluye el tamaño total del elemento, incluido el relleno y el borde (pero no el margen).
- Utiliza un color de fondo y un color de texto que aseguren que el texto sea legible.

Para más información, ver

http://www.w3schools.com/cssref/pr_background-color.asp

Tipografía (font-family, font-size)



Las propiedades font-family y font-size son las herramientas esenciales para controlar la apariencia fundamental de tu texto: **el tipo de letra y su tamaño.**

font-family determina la fuente tipográfica utilizada, permitiéndote elegir entre fuentes instaladas en el usuario o fuentes web.

font-size establece cuán grande o pequeño se muestra el texto.

Juntas, estas propiedades definen la legibilidad y la jerarquía básica de tu contenido textual.

SINTAXIS

font-family: family-name1 , family-name2, generic-family;

/ Puedes especificar múltiples family-name seguidos de una generic-family */*

Fuentes múltiples:

```
h1{  
  
  font-family: Arial, Helvetica, "Times New Roman", sans-serif;  
  
}
```

Si el nombre de una fuente tiene espacios, debe ir entre comillas (ej: "Times New Roman").



CONSEJOS

- Puedes especificar una lista de fuentes, de mayor a menor prioridad. Si el navegador no admite la primera fuente, intentará usar la siguiente.
- Siempre termina tu lista de font-family con una familia genérica (serif, sans-serif, monospace, cursive, o fantasy) como última opción de respaldo.

Para más información, ver
https://www.w3schools.com/css/css_font.asp



La propiedad **font-size** controla el **tamaño del texto** dentro de un elemento.

Es una propiedad muy común para ajustar la legibilidad y la jerarquía visual de los elementos textuales.

Puedes usar diversas unidades para definir el tamaño, como:

- **Absolutas: px (píxeles)** - tamaño fijo.
- **Relativas al elemento padre:** em - relativo al tamaño de fuente del elemento padre.
- **Relativas al elemento raíz (html):** rem - relativo al tamaño de fuente del elemento html.
- **Relativas al viewport:** vw (viewport width), vh (viewport height) - relativo al tamaño de la ventana del navegador.
- **Palabras clave:** small, medium, large, x-large, etc., o smaller, larger (relativos al tamaño de fuente del padre).

SINTAXIS

font-size: value;

/ value puede ser una medida (px, em, rem, %), o una palabra clave */*

trinket

Autorun

Share

Save

index.html

styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10  <p class="texto-con-fuente">Este párrafo debería usar 'Arial' o, si no está
    disponible, 'Helvetica', y finalmente cualquier fuente 'sans-serif'.
11  </p>
12  <p class="size-px">Texto con tamaño en píxeles (16px por defecto del navegador)
    </p>
13  <p class="size-em">Texto con tamaño en em (1.2em)</p>
14  <p class="size-rem">Texto con tamaño en rem (1.5rem)</p>
15
16 </body>
17 </html>
```

Result

Instructions

Prueba tú mismo:

Cambia la lista de fuentes. Prueba a poner Times New Roman, Georgia, serif. Observa el cambio en el estilo del texto.

Intenta poner una fuente que sabes que no está en tu sistema (ej: font-family: "Una Fuente Inventada", Verdana, sans-serif;). ¿Qué fuente se usa?

Busca información sobre Google Fonts y cómo incluirlas en tu proyecto para usar fuentes web que no dependen de las fuentes instaladas en el usuario.

Descomenta la regla html { font-size: 10px; } en el CSS. Observa cómo cambian los tamaños de los párrafos con em y rem (ahora 1.2em serían 12px y 1.5rem serían 15px), mientras que el párrafo con px permanece igual.

Explicación del Ejemplo:

- **.size-px:** Establece el tamaño de fuente a 20px, un valor absoluto que no cambia con el tamaño de fuente del padre o la raíz.
- **.size-em:** Establece el tamaño de fuente a 1.2em. Este tamaño es 1.2 veces el tamaño de fuente del elemento *padre*. Si el párrafo está directamente en el body (cuyo tamaño por defecto es 16px en la mayoría de los navegadores), 1.2em serán 19.2px.

Si este párrafo estuviera dentro de un div con font-size: 20px;, entonces 1.2em serían $20 * 1.2 = 24\text{px}$.

- **.size-rem:** Establece el tamaño de fuente a 1.5rem. Este tamaño es 1.5 veces el tamaño de fuente del elemento *raíz* (<html>). El tamaño de fuente del <html> suele ser 16px por defecto. 1.5rem serían $16 * 1.5 = 24\text{px}$.

Usar rem es útil porque hace que el tamaño de fuente sea predecible sin importar la anidación de los elementos.



CONSEJOS

- El valor predeterminado del tamaño de fuente es medium (generalmente 16px en navegadores).
- Las unidades em y rem son útiles para crear tipografías responsivas, ya que se escalan en relación con otros elementos o el tamaño base. rem es a menudo más fácil de manejar que em debido a su relación constante con el elemento raíz.

Para más información, ver https://www.w3schools.com/css/css_font.asp

color

Ceibal 

Propiedad:
color

La propiedad color es muy sencilla pero esencial: **define el color del texto que aparece dentro de un elemento.**

Es importante distinguirla de background-color, que pinta el fondo.

Al igual que con background-color, puedes especificar el color usando nombres de colores predefinidos, valores hexadecimales (#RRGGBB) o valores RGB (rgb(rojo, verde, azul)).

Asegurar un buen contraste entre el color del texto y el color de fondo es crucial para la accesibilidad y legibilidad.

SINTAXIS

color: value;

/ value puede ser un nombre, HEX, RGB, HSL, etc. */*

Color en HEX:

```
h1{  
  
  color: #92a8d1;  
  
}
```

Color en RGB:

```
h1{  
  
  color: rgb(220, 20, 60);  
  
}
```

Nombre del color:

```
p{  
  
  color: green;  
  
}
```

trinket

Autorun

Share

index.html styles.css

1

<!DOCTYPE html>

2

<html lang="es">

3

<head>

4

<meta charset="UTF-8">

5

<meta name="viewport" content="width=device-width, initial-scale=1.0">

6

<title>Ejemplo Border </title>

7

<link rel="stylesheet" href="styles.css"/>

8

</head>

9

<body>

10

<h1 class="titulo-especial">Título con color personalizado</h1>

11

<p class="parrafo-especial">Este texto también tiene un color diferente.</p>

12

</body>

13

</html>

Result

Instructions

Prueba tú mismo:

Cambia el color del texto usando un nombre de color (ej: orange) en lugar de HEX o RGB.

Aplica un background-color a uno de los elementos y asegúrate de que haya suficiente contraste con el color del texto para que sea fácil de leer.

Crea una lista (< ul>) y aplica un color al ul. ¿Los elementos < li> dentro heredan el color? (Sí, la propiedad color es típicamente heredada por los elementos hijos).

Explicación del Ejemplo:

Al encabezado h1 con la clase titulo-especial, le aplicamos: **color: #0077b6;**

Lo que cambia el color de su texto a ese tono de azul. Al párrafo p con la clase parrafo-especial, le aplicamos:

color: rgb(128, 0, 128);

Cambiando su texto a púrpura.



CONSEJOS

- Asegúrate de elegir un color de fondo que combine con el color del texto para garantizar que sea legible.
- Considera la accesibilidad (WCAG) para usuarios con discapacidades visuales.

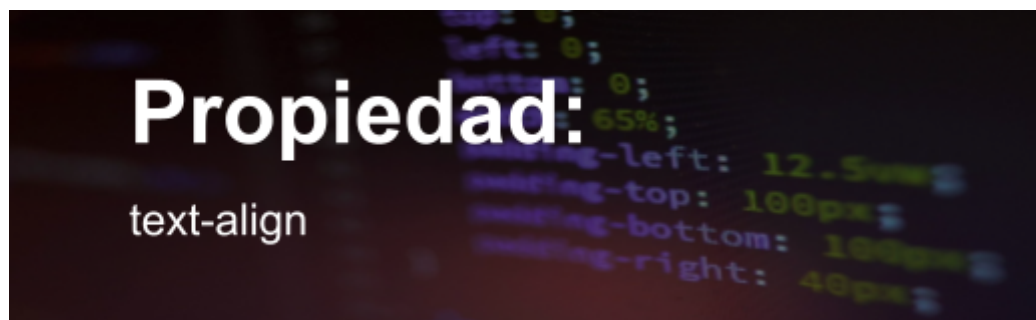
Para más información, ver

<http://www.w3schools.com/cssref/prtextcolor.asp>

text-align



Ceibal



La propiedad **text-align** controla la alineación horizontal del contenido en línea (principalmente texto y elementos en línea como `<span>` o `<img>`) dentro del elemento a nivel de bloque que los contienen. Es muy útil para centrar bloques de texto o justificar párrafos.

Los valores más comunes son **left**, **right**, **center** y **justify**.

SINTAXIS

text-align: value;

La propiedad `text-align` especifica la alineación horizontal del contenido en línea (principalmente texto) dentro de un elemento de bloque.

trinket

Autorun

Share

index.htmlstyles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10
11
12 <h1>Ejemplos de Alineación de Texto</h1>
13
14 <div class="text-block align-left">
15   <h2>Alineado a la Izquierda</h2>
16   <p>Este texto está alineado a la izquierda. Esta es la alineación por defecto y generalmente la mejor para bloques largos de lectura en idiomas que se leen de izquierda a derecha.</p>
17 </div>
18
19 <div class="text-block align-center">
20   <h2>Centrado</h2>
21   <p>Este texto está centrado. Es útil para títulos o citas cortas, pero no recomendado para párrafos largos.</p>
22 </div>
23
24 <div class="text-block align-right">
25   <h2>Alineado a la Derecha</h2>
26   <p>Este texto está alineado a la derecha. Se usa menos comúnmente en layouts de izquierda a derecha, a menudo para elementos específicos o diseños creativos.</p>
27 </div>
28
29 <div class="text-block align-justify">
30   <h2>Justificado</h2>
31   <p>Este texto está justificado. Se ajusta el espacio entre palabras para que ambos bordes, izquierdo y derecho, estén alineados. Esto puede crear un bloque visualmente uniforme, pero a veces introduce espacios grandes entre palabras si el texto no llena la línea.</p>
32 </div>
33
34 </body>
35 </html>
```

Result

Instructions

Ejemplos de Alineación de Texto

Alineado a la Izquierda

Este texto está alineado a la izquierda. Esta es la alineación por defecto y generalmente la mejor para bloques largos de lectura en idiomas que se leen de izquierda a derecha.

Centrado

Este texto está centrado. Es útil para títulos o citas cortas, pero no recomendado para párrafos largos.

Alineado a la Derecha

Este texto está alineado a la derecha. Se usa menos comúnmente en layouts de izquierda a derecha, a menudo para elementos específicos o diseños creativos.

Justificado

Este texto está justificado. Se ajusta el espacio entre palabras para que ambos bordes, izquierdo y derecho, estén alineados. Esto puede crear un bloque visualmente uniforme, pero a veces introduce espacios grandes entre palabras si el texto no llena la línea.

Explicación del Ejemplo:

Aplicamos diferentes valores de text-align a cada párrafo dentro del contenedor:

- **.align-left:** El texto se alinea con el borde izquierdo del contenedor. Este es el comportamiento por defecto para el texto en la mayoría de los navegadores y elementos.
- **.align-center:** El texto se centra horizontalmente dentro del contenedor.
- **.align-right:** El texto se alinea con el borde derecho del contenedor.
- **.align-justify:** El texto se extiende a lo largo de todo el ancho del contenedor, distribuyendo el espacio extra entre las palabras para que ambos bordes (izquierdo y derecho) queden alineados.

text-decoration



Ceibal



La propiedad **text-decoration** se utiliza para añadir o remover líneas decorativas del texto.

Los usos más comunes son el subrayado (underline), el sobrelineado (overline) o el tachado (line-through).

Es especialmente útil para quitar el subrayado por defecto de los enlaces (text-decoration: none;).

La propiedad text-decoration es una abreviatura que puede incluir el tipo de línea, el color, el estilo de la línea (como wavy o dashed), y el grosor.

SINTAXIS

text-decoration: value;

none | underline || overline || line-through || <'text-decoration-line'> <'text-decoration-color'>
<'text-decoration-style'> <'text-decoration-thickness'>;

Puedes combinar underline, overline, line-through.

Los otros valores (color, estilo, grosor) se refieren a sub-propiedades.

The screenshot shows the Trinket online code editor interface. On the left, the code editor has two tabs: 'index.html' and 'styles.css'. The 'index.html' tab is active, showing the following code:

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10
11 <p class="underline">Este texto está subrayado.</p>
12 <p class="overline">Este texto está sobrelineado.</p>
13 <p class="line-through">Este texto está tachado.</p>
14 <a href="#" class="no-underline">Este enlace no está subrayado por defecto.</a>
15 <h2 class="multi-decoration">Título con múltiples decoraciones</h2>
16
17 </body>
18 </html>
```

The 'styles.css' tab is also visible but empty. On the right, the 'Result' tab shows the rendered output of the code:

Este texto está subrayado.

Este texto está sobrelineado.

Este texto está tachado.

Este enlace no está subrayado por defecto.

Título con múltiples decoraciones

Explicación del Ejemplo:

.underline, .overline, .line-through: Aplican los estilos de línea correspondientes (subrayado, sobrelineado, tachado).

.no-underline: Usa text-decoration: none; para eliminar cualquier decoración, lo que es común para los enlaces (<a>) que vienen con un subrayado por defecto.

.multi-decoration: Demuestra que text-decoration puede combinar tipo, color y estilo. La línea azul ondulada aparece debajo del título. (Nota: la especificación más reciente permite el grosor directamente, pero históricamente era con text-decoration-thickness).



CONSEJOS

- La propiedad `text-decoration` especifica la apariencia de las líneas decorativas en el texto.
- Es una propiedad abreviada para `text-decoration-line`, `text-decoration-color`, `text-decoration-style` y `text-decoration-thickness`.
- Para más información, ver https://www.w3schools.com/css/css_text_decoration.asp

border



Ceibal

Propiedad: border

La propiedad **border** es una propiedad abreviada muy útil que te permite definir rápidamente el estilo, el ancho y el color de los cuatro bordes de un elemento en una sola declaración CSS.

Esto es más eficiente y conciso que definir border-width, border-style y border-color por separado para cada lado cuando los cuatro bordes son iguales.

SINTAXIS

border: <border-width> <border-style> <border-color>;

Propiedad de Borde:

```
h1{
```

```
border: 5px solid red;
```

```
}
```

Autorun
Share
Save

index.html

styles.css

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10   <h2>Este encabezado tiene un borde.</h2>
11
12 </body>
13 </html>
```

Result

Instructions

Prueba tú mismo:

- Cambia el estilo del borde a dashed, dotted, double o groove. ¿Cómo se ve el borde?
- Modifica el ancho del borde a un valor más grande (ej: 10px) o más pequeño (ej: 1px).
- Prueba a usar un valor diferente para el color, como un HEX o RGB.
- Elimina la propiedad border-style (ej: deja solo border: 5px red;). ¿Qué ocurre y por qué?



CONSEJOS

- Si no se proporciona el valor `border-style`, el valor predeterminado es `none`, por lo que el borde no será visible. ¡Siempre incluye el estilo!
- Si no se especifica `border-color`, usará el color del texto del elemento.
- Si no se establece `border-width`, el valor predeterminado es `medium`.

Para más información, ver

https://www.w3schools.com/css/css_border.asp

border-bottom, border-left, border-right, border-top



Propiedad:
border-bottom,
border-left, border-right,
border-top

A diferencia de la propiedad abreviada `border` que afecta a los cuatro lados, estas propiedades te permiten controlar el estilo, el ancho y el color de un borde específico: el inferior (`border-bottom`), el izquierdo (`border-left`), el derecho (`border-right`) o el superior (`border-top`).

Son especialmente útiles cuando necesitas que los bordes de un elemento tengan apariencias diferentes en cada lado, o si solo quieres un borde en uno o dos lados.

SINTAXIS

border-[side]: border-width border-style border-color;

[side]=inferior (border-bottom), el izquierdo (border-left), el derecho (border-right) o el superior (border-top).

Similar a border, el orden de los valores generalmente no importa, pero width style color es el estándar.

Propiedad de Borde inferior:

```
h1{  
  
border-bottom: 5px solid red;  
  
}
```



Explicación del Ejemplo:

Aplicamos la propiedad border-bottom al párrafo. Le indicamos que el borde inferior debe tener un grosor de 4px, un estilo dotted (punteado) y ser de color green.

Los otros tres bordes del párrafo (arriba, izquierda, derecha) no tienen borde por defecto.



CONSEJOS

- Si no se proporciona `border-style`, el valor predeterminado es `none`, haciendo el borde invisible.
- Si no se especifica `border-color`, usará el color del texto del elemento.
- Si no se establece `border-width`, el valor predeterminado es `medium`.

Para más información, ver

https://www.w3schools.com/css/css_border.asp

Bordes color



Ceibal



La propiedad border-color te permite establecer el color de los bordes de un elemento.

Es importante recordar que esta propiedad *solo* define el color; para que un borde **sea visible, también debes definir su estilo (border-style)**.

Puedes especificar un único color para los cuatro lados, o diferentes colores para cada lado en un orden específico: superior, derecho, inferior, izquierdo (sentido horario).

También puedes usar 2 o 3 valores para aplicar colores a pares de lados o a tres lados respectivamente.

SINTAXIS

border-color: value; /* Un valor: aplica a los 4 lados */

border-color: value1 value2; /* Dos valores: aplica (Top/Bottom) (Left/Right) */

border-color: value1 value2 value3; /* Tres valores: aplica (Top) (Left/Right) (Bottom) */

border-color: value1 value2 value3 value4; /* Cuatro valores: aplica (Top) (Right) (Bottom) (Left) */

Bordes rojos:

```
h1{  
  
  border-style: solid;  
  
  border-color: red;  
  
}
```

Bordes de colores diferentes:

```
h1{  
  
  border-style: solid;  
  
  border-color: red green blue purple;  
  
}
```

trinket

Autorun

Share

Viewing DraftSave

index.htmlstyles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10
11 <div class="caja-colores-borde">
12   Esta caja tiene bordes de diferentes colores.
13 </div>
14
15
16 </body>
17 </html>
```

Result

Instructions

Prueba tú mismo:

Cambia border-color a un solo valor (ej: border-color: purple;). ¿Qué ocurre con los bordes?

Prueba con dos valores (ej: border-color: hotpink teal;). El primer valor se aplica arriba y abajo, el segundo a izquierda y derecha.

Prueba con tres valores (ej: border-color: brown yellow cyan;). El primer valor es para arriba, el segundo para izquierda y derecha, el tercero para abajo.

Intenta definir los colores usando valores HEX o RGB en lugar de nombres.

Explicación del Ejemplo:

Primero, nos aseguramos de que el div tenga un estilo (solid) y un ancho (5px) para que los bordes sean visibles.

Luego, con border-color: red green blue orange;, esto asigna los colores en el siguiente orden:

- Borde superior (top): red
- Borde derecho (right): green
- Borde inferior (bottom): blue
- Borde izquierdo (left): orange



CONSEJOS

- Siempre declara la propiedad border-style antes de border-color (o usa la abreviada border) para asegurarte de que el borde sea visible.

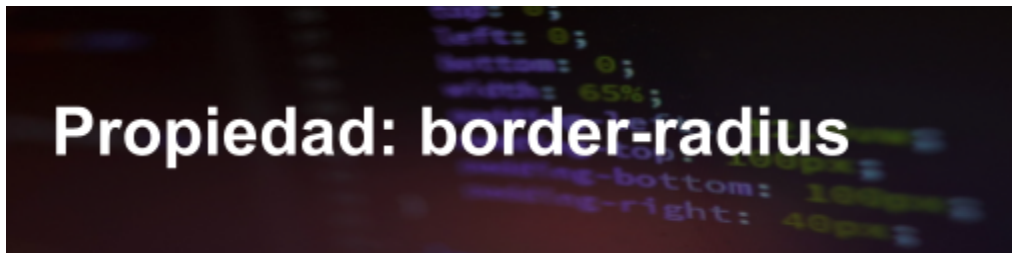
Para más información, ver

https://www.w3schools.com/css/css_border.asp

Bordes redondeados



Ceibal



La propiedad `border-radius` es una herramienta poderosa para suavizar las esquinas cuadradas predeterminadas de un elemento, dándoles una forma redondeada.

Puedes aplicar el mismo radio a las cuatro esquinas o especificar radios diferentes para cada una, creando formas de caja muy interesantes.

Los valores pueden ser en píxeles (px), porcentajes (%), u otras unidades de medida.

SINTAXIS

/ Un valor: aplica el mismo radio a las 4 esquinas */*

border-radius: value;

/ Dos valores: aplica (Top-Left & Bottom-Right) (Top-Right & Bottom-Left) */*

border-radius: value1 value2;

/ Tres valores: aplica (Top-Left) (Top-Right & Bottom-Left) (Bottom-Right) */*

border-radius: value1 value2 value3;

/ Cuatro valores: aplica (Top-Left) (Top-Right) (Bottom-Right) (Bottom-Left) */*

border-radius: value1 value2 value3 value4;

/ Para radios elípticos, usa el formato horizontal / vertical */*

border-radius: h-value1 h-value2 h-value3 h-value4 / v-value1 v-value2 v-value3 v-value4;

border-radius: h-value / v-value; */* shorthand */*

Bordes redondeados en píxeles:

h1 {

border: 2px solid red;

border-radius: 25px;

}

Bordes redondeados en porcentaje:

```
h1{  
  
border: 2px solid red;  
  
border-radius: 20%;  
  
}
```



Explicación del Ejemplo:

Aplicamos un borde simple a la imagen para visualizar el efecto.

Luego, con `border-radius: 15px;`, indicamos que las cuatro esquinas de la imagen (y su borde) deben redondearse con un radio de 15 píxeles. **Esto crea esquinas suaves.**



CONSEJOS

- Puedes aplicar `border-radius` a cualquier elemento con o sin borde. Si no tiene borde, el redondeo se aplica al borde del área de fondo y contenido.
- El valor predeterminado es 0 (sin redondeo).
- La forma de las esquinas puede definirse en píxeles, porcentajes u otras unidades.

Para más información, ver https://www.w3schools.com/cssref/css3_pr_border-radius.php

Bordes estilos



Propiedad: border-style

La propiedad border-style es crucial para que un borde sea visible, ya que define la apariencia visual de la línea del borde.

Sin un estilo definido (su valor predeterminado es none), incluso si defines el ancho y el color, el borde no se mostrará.

Hay varios estilos disponibles como solid (línea continua), dashed (guiones), dotted (puntos), double (doble línea), groove (efecto 3D de ranura), ridge (efecto 3D de cresta), inset (efecto 3D de hundido) y outset (efecto 3D de salido).

Puedes aplicar el mismo estilo a los cuatro lados o diferentes estilos a cada lado de manera similar a border-color.

SINTAXIS

border-style: value; /* Un valor: aplica a los 4 lados */

border-style: value1 value2;

/* Dos valores: aplica (Top/Bottom) (Left/Right) */

border-style: value1 value2 value3;

/* Tres valores: aplica (Top) (Left/Right) (Bottom) */

border-style: value1 value2 value3 value4;

/* Cuatro valores: aplica (Top) (Right) (Bottom) (Left) */

Bordes redondeados en píxeles:

```
h1{  
  
border: 2px solid red;  
  
border-radius: 25px;  
  
}
```

Bordes redondeados en porcentaje:

```
h1{  
  
border: 2px solid red;  
  
border-radius: 20%;  
  
}
```

trinket

Autorun

Share

index.html styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10  <h1>Ejemplo de todos los estilos de borde</h1>
11
12  <div class="border-example solid">Borde sólido</div>
13  <div class="border-example dotted">Borde punteado</div>
14  <div class="border-example dashed">Borde discontinuo</div>
15  <div class="border-example double">Borde doble</div>
16  <div class="border-example groove">Borde en surco</div>
17  <div class="border-example ridge">Borde en relieve</div>
18  <div class="border-example inset">Borde insertado</div>
19  <div class="border-example outset">Borde sobresaliente</div>
20  <div class="caja-estilos-borde">
21    Bordes con diferentes estilos.
22  </div>
23
24 </body>
25 </html>
```

Result

Instructions

Prueba tú mismo:

Cambia border-style a un solo valor (ej: border-style: groove;). ¿Qué ocurre con los bordes? (Nota: los estilos 3D como groove, ridge, inset, outset a veces requieren un color y/o ancho para verse mejor).

Prueba con dos valores (ej: border-style: ridge inset;). El primer valor se aplica arriba y abajo, el segundo a izquierda y derecha.

Prueba con tres valores (ej: border-style: solid double dotted;). El primer valor es para arriba, el segundo para izquierda y derecha, el tercero para abajo.

Aplica solo border-style: dashed; a un elemento sin especificar border-width ni border-color. ¿Es visible y por qué?

Explicación del Ejemplo:

Especificamos un ancho (3px) y un color (navy) para los bordes. Luego, con:

border-style: dotted solid dashed double;

Definimos los estilos para cada lado:

- Borde superior (top): dotted
- Borde derecho (right): solid
- Borde inferior (bottom): dashed
- Borde izquierdo (left): double



CONSEJOS

- Puedes aplicar border-radius a cualquier elemento con o sin borde. Si no tiene borde, el redondeo se aplica al borde del área de fondo y contenido.
- El valor predeterminado es 0 (sin redondeo).
- La forma de las esquinas puede definirse en píxeles, porcentajes u otras unidades.

Para más información, ver

https://www.w3schools.com/cssref/css3_pr_border-radius.php

Bordes - ancho

Ceibal

Ceibal



Propiedad: border-width

La propiedad border-width te permite controlar el grosor de los bordes de un elemento.

Al igual que con border-color y border-style, para que el ancho tenga efecto, el borde debe tener un estilo definido (no none).

Puedes usar palabras clave predefinidas como **thin, medium o thick**, o **especificar un valor numérico con una unidad (como px, em, rem)**.

Similar a otras propiedades de borde, puedes establecer un ancho único para los cuatro lados o especificar anchos diferentes para cada lado en el orden Top, Right, Bottom, Left.

SINTAXIS

/ Un valor: aplica a los 4 lados */*

**border-width: value; /* value puede ser thin, medium, thick,
o una medida (ej: 5px) */**

/ Dos valores: aplica (Top/Bottom) (Left/Right) */*

border-width: value1 value2;

/ Tres valores: aplica (Top) (Left/Right) (Bottom) */*

border-width: value1 value2 value3;

/ Cuatro valores: aplica (Top) (Right) (Bottom) (Left) */*

border-width: value1 value2 value3 value4;

Ancho del borde con valor predefinido:

```
h1 {  
  
  border-style: solid;  
  
  border-width: thick;  
  
}
```

Ancho del borde con valores diferentes para cada lado:

```
h1 {  
  
  border-style: solid;  
  
  border-width: 25px 10px 4px 35px;  
  
}
```

trinket

Autorun

Share

index.html

styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10 <div class="caja-anchos-borde">
11   Bordes con diferentes grosores.
12 </div>
13
14
15 </body>
16 </html>
```

Result

Instructions

Prueba tú mismo:

Cambia border-width a un solo valor (ej: border-width: thick;). ¿Cómo se ve el borde ahora?

Prueba con dos valores (ej: border-width: 1px 5px;). El primer valor se aplica arriba y abajo, el segundo a izquierda y derecha.

Prueba con tres valores (ej: border-width: thin medium thick;). El primer valor es para arriba, el segundo para izquierda y derecha, el tercero para abajo.

Aplica un border-width a un elemento sin especificar border-style (o con border-style: none;). ¿Es visible el borde?

Explicación del Ejemplo:

Primero, definimos un estilo (solid) y un color (blue) para los bordes. Luego, con :

border-width: 25px 10px 5px 20px;

Asignamos los grosores:

- Borde superior (top): 25px
- Borde derecho (right): 10px
- Borde inferior (bottom): 5px
- Borde izquierdo (left): 20px



CONSEJOS

- El valor predeterminado del ancho del borde es **medium**.
- Puedes especificar valores de ancho del borde para cada uno de los lados de un elemento (superior, derecho, inferior e izquierdo) usando 1, 2, 3 o 4 valores.

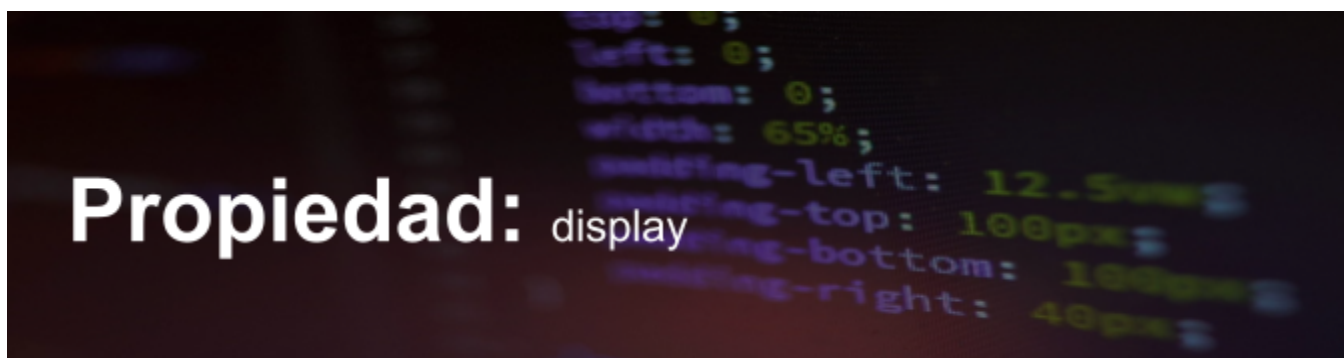
Para más información, ver

http://www.w3schools.com/cssref/pr_border-width.asp

display



Ceibal



La propiedad display es, la propiedad CSS más importante para controlar el layout o la disposición de los elementos en una página web. Se utiliza para especificar cómo debe mostrarse un elemento en la pantalla.

Cada elemento HTML viene con un valor display predeterminado que depende del tipo de elemento que sea. Los valores por defecto más comunes para la mayoría de los elementos son block o inline. La propiedad display en CSS nos permite cambiar este comportamiento predeterminado.

Elementos de Nivel de Bloque (block)

Un elemento de nivel de bloque siempre comienza en una nueva línea en la página.

Además, por defecto, ocupa todo el ancho disponible que su contenedor le permite (se extiende hacia la izquierda y hacia la derecha todo lo posible).

Ejemplos comunes de elementos de nivel de bloque incluyen:

- `<div>`
- `<h1>` a `<h6>`
- `<p>`

- **<form>**
- **<header>**
- **<footer>**
- **<section>**

Elementos en Línea (inline)

A diferencia de los elementos de bloque, un elemento en línea **no** comienza en una nueva línea.

Solo ocupa el ancho necesario para mostrar su contenido. Fluyen dentro del texto o el contenido de otros elementos.

Ejemplos comunes de elementos de nivel en línea incluyen:

- ****
- **<a>**
- ****

Valores de la Propiedad display

La propiedad display tiene una variedad de valores que nos permiten alterar el comportamiento de visualización por defecto de los elementos. Algunos de los más importantes y comunes son:

- **inline:** Muestra un elemento como un elemento en línea.
- **block:** Muestra un elemento como un elemento de nivel de bloque.
- **inline-block:** Muestra un elemento como un bloque, permitiendo aplicar propiedades como width y height, pero sin que comience una nueva línea. El elemento en sí se formatea como un elemento en línea dentro de su flujo.
- **none:** Elimina completamente el elemento de la página. El espacio que ocupaba ya no se considera en el layout.
- **flex:** Muestra un elemento como un contenedor flexible a nivel de bloque. (Fundamental para Flexbox)
- **grid:** Muestra un elemento como un contenedor de cuadrícula a nivel de bloque. (Fundamental para CSS Grid)
- **contents:** Hace que el contenedor desaparezca, convirtiendo a sus hijos en hijos

directos del elemento inmediatamente superior en el DOM.

- Otros valores incluyen `list-item`, `table`, `table-cell`, etc., que hacen que un elemento se comporte como elementos de tabla o de lista.



El valor **`display: none;`** es particularmente útil para ocultar elementos. Cuando se aplica a un elemento, este es completamente removido del layout de la página. Es como si el elemento no existiera en absoluto para el propósito de la disposición de los elementos.

Esto es diferente de **`visibility: hidden;`**, que también oculta un elemento, pero deja el espacio que ocuparía en el layout, como si fuera un espacio vacío.

Anulando el Comportamiento `display` por Defecto

Como se mencionó, cada elemento tiene un valor `display` predeterminado. Sin embargo, puedes anularlo con CSS.

Cambiar un elemento en línea a bloque o viceversa puede ser muy útil para lograr layouts específicos.

Un ejemplo común es hacer que los elementos de lista (`<li>`), que por defecto son de nivel de bloque (aunque suelen mostrarse con marcadores y cierto indentado que los hace parecer diferentes a un `div`), se comporten como elementos en línea o `inline-block` para crear menús horizontales.

En resumen, la propiedad `display` es fundamental para controlar si un elemento comienza una nueva línea, cuánto ancho ocupa y cómo interactúa con los elementos que lo rodean en el flujo del documento.

Dominar `block`, `inline` e `inline-block` es clave para la maquetación básica, mientras que `none`, `flex` y `grid` abren la puerta a técnicas de layout más avanzadas.

trinket

Autoun

Share

Save

Instructions

index.html

styles.css

```
3 <!--
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Demostración Visual de Propiedad Display</title>
7 <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10
11 <div class="container">
12   <h1>Entendiendo la Propiedad 'display' en CSS</h1>
13   <p>La propiedad <code>display</code> es fundamental para controlar cómo se muestra y se comporta un elemento en el layout
14   de la página. Observa las diferencias entre los tipos principales.</p>
15
16   <div class="demo-section">
17     <h2>Display por Defecto: 'block'</h2>
18     <p>Los elementos block siempre inician en una nueva línea y, por defecto, ocupan todo el ancho disponible de su
19     contenedor.</p>
20     <div class="display-box block-demo">Este es un DIV (Block por defecto).</div>
21     <div class="display-box block-demo">Este es otro DIV (Block por defecto).</div>
22     <p>O <code>class="display-box block-demo">Este es un Párrafo (P, también Block por defecto).</p>
23   </div>
24
25   <div class="demo-section">
26     <h2>Display por Defecto: 'inline'</h2>
27     <p>Los elementos inline no inician en una nueva línea y solo ocupan el ancho necesario para su contenido. No respetan
28     <code>width</code> ni <code>height</code> explícitos y el <code>margin</code> <code>padding</code> vertical puede
29     afectar la línea de texto.</p>
30     <span class="display-box inline-demo">Este es un SPAN (Inline por defecto).</span>
31     <span class="display-box inline-demo">Este es otro SPAN (Inline por defecto).</span>
32     <a href="#" class="display-box inline-demo">Este es un Enlace A (Inline por defecto).</a>
33     <p>Texto que continúa el flujo.</p>
34   </div>
35
36   <div class="demo-section">
37     <h2>Cambiando Display: 'inline-block'</h2>
38     <p>Los elementos con <code>display: inline-block</code> se comportan como elementos inline (no fuerzan nueva línea),
39     pero respetan <code>width</code>, <code>height</code>, y todos los márgenes/rellenos, como si fueran bloques.
40     Ideal para elementos de cuadrícula simples o botones con dimensiones.</p>
41     <div class="display-box inline-block-demo">DIV con display: inline-block.</div>
42     <div class="display-box inline-block-demo">Otro DIV con display: inline-block.</div>
43     <span class="display-box inline-block-demo">SPAN con display: inline-block.</span>
44     
45   </div>
46
47   <div class="demo-section">
48     <h2>Cambiando Display: 'block' a 'inline' y 'inline' a 'block'</h2>
49     <p>Podemos cambiar el comportamiento por defecto si lo necesitamos.</p>
50     <div class="display-box block-to-inline">Este DIV (normalmente Block) ahora es display: inline.</div>
51     <p>Texto después del DIV inline.</p>
52     <span class="display-box inline-to-block">Este SPAN (normalmente Inline) ahora es display: block.</span>
53     <p>Texto después del SPAN block.</p>
54   </div>
55
56   <div class="demo-section">
57     <h2>Ocultando Elementos: 'display: none;' vs 'visibility: hidden;'</h2>
58     <p>Ambos ocultan elementos, pero de forma diferente.</p>
59     <div class="display-box normal-box">Este es un elemento normal antes de uno oculto.</div>
60     <div class="display-box hidden-normal-box">Este elemento tiene display: none.</div>
61     <div class="display-box normal-box">Este elemento está después de uno con display: none; Nota que el espacio del
62     elemento oculto no existe.</div>
63
64     <div class="display-box normal-box">Este es un elemento normal antes de uno oculto con visibility.</div>
65     <div class="display-box hidden-visibility-demo">Este elemento tiene visibility: hidden.</div>
66     <div class="display-box normal-box">Este elemento está después de uno con visibility: hidden; Nota que el espacio
67     del elemento oculto SÍ existe, está vacío.</div>
68   </div>
69
70 </body>
71 </html>
```

Result

Entendiendo la Propiedad 'display' en CSS

La propiedad `display` es fundamental para controlar cómo se muestra y se comporta un elemento en el layout de la página. Observa las diferencias entre los tipos principales.

Display por Defecto: 'block'

Los elementos block siempre inician en una nueva línea y, por defecto, ocupan todo el ancho disponible de su contenedor

Este es un DIV (Block por defecto).

Este es otro DIV (Block por defecto).

Este es un Párrafo (P, también Block por defecto).

Display por Defecto: 'inline'

Los elementos inline no inician en una nueva línea y solo ocupan el ancho necesario para su contenido. No respetan `width` ni `height` explícitos y el `margin` / `padding` vertical puede afectar la línea de texto.

Este es un SPAN (inline por defecto).

Este es otro SPAN (inline por defecto).

Este es un Enlace A (inline por defecto).

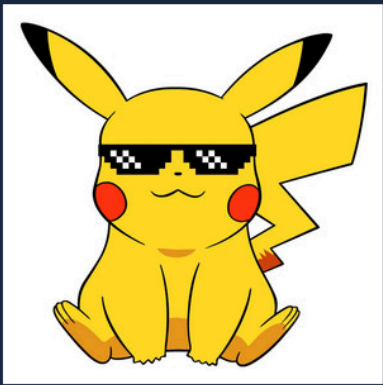
Display por Defecto: 'inline-block'

Los elementos con `display: inline-block`; se comportan como elementos inline (no fuerzan nueva línea), pero respetan `width`, `height`, y todos los márgenes/rellenos, como si fueran bloques. Ideal para elementos de cuadrícula simples o botones con dimensiones.

DIV con display: inline-block;

DIV con display: inline-block;

Otro DIV con display: inline-block;



SPAN con display: inline-block;

Cambiando Display: 'block' a Inline y 'inline' a Block

Podemos cambiar el comportamiento por defecto si lo necesitamos.

Este DIV (normalmente Block) ahora es display: inline;.

Texto después del DIV inline.

Este SPAN (normalmente Inline) ahora es display: block;.

Texto después del SPAN block.

Ocultando Elementos: 'display: none;' vs 'visibility: hidden;'

Ambos ocultan elementos, pero de forma diferente.

Este es un elemento normal antes de uno oculto.

Este elemento tiene display: none;

Este elemento está después de uno con display: none;. Nota que el espacio del elemento oculto no existe.

Este es un elemento normal antes de uno oculto con visibility.

Este elemento tiene visibility: hidden;

Este elemento está después de uno con visibility: hidden;. Nota que el espacio del elemento oculto Sí existe, está vacío.

Explicación del Ejemplo:

.underline, .overline, .line-through: Aplican los estilos de línea correspondientes (subrayado, sobrelineado, tachado).

.no-underline: Usa `text-decoration: none;` para eliminar cualquier decoración, lo que es común para los enlaces (`<a>`) que vienen con un subrayado por defecto.

.multi-decoration: Demuestra que `text-decoration` puede combinar tipo, color y estilo. La línea azul ondulada aparece debajo del título. (Nota: la especificación más reciente permite el grosor directamente, pero históricamente era con `text-decoration-thickness`).



CONSEJOS

- La propiedad `text-decoration` especifica la apariencia de las líneas decorativas en el texto.
- Es una propiedad abreviada para `text-decoration-line`, `text-decoration-color`, `text-decoration-style` y `text-decoration-thickness`.
- Para más información, ver https://www.w3schools.com/css/css_text_decoration.asp

float

Ceibal



Propiedad: float

La propiedad **float** se utiliza principalmente en diseño de layouts para posicionar un elemento (como una imagen o una caja de texto) a la izquierda o derecha dentro de su contenedor.

Cuando un elemento "flota", se saca del flujo normal del documento, y el contenido que le sigue (como texto) se envuelve alrededor de él.

Es una técnica común, aunque las modernas herramientas de layout como **Flexbox y Grid** son a menudo preferibles para estructuras de página más complejas.

SINTAXIS

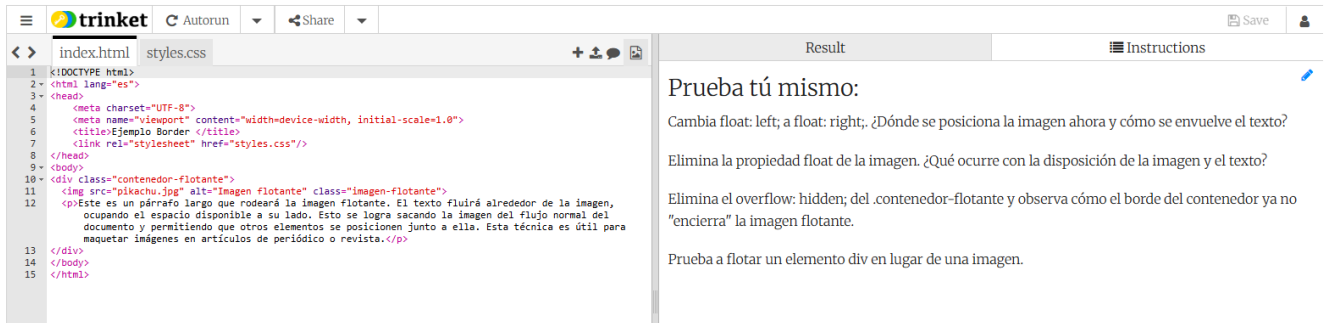
float: value;

value= none | left | right | initial | inherit;

Imagen flotante a la derecha:

```
img {  
  
  float: right;  
  
}
```

Importante: Usar `overflow: hidden;` o una pseudoclase `::after` con `clear: both;` en el contenedor para "limpiar" el float y asegurar que el borde del contenedor envuelva el contenido flotado y el texto



The screenshot shows the Trinket web editor interface. On the left, the code editor displays the following HTML and CSS:

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10  <div class="contenedor-flotante">
11    
12    <p>Este es un párrafo largo que rodeará la imagen flotante. El texto fluirá alrededor de la imagen,
    ocupando el espacio disponible a su lado. Esto se logra sacando la imagen del flujo normal del
    documento y permitiendo que otros elementos se posicionen junto a ella. Esta técnica es útil para
    maquetar imágenes en artículos de periódico o revista.</p>
13  </div>
14 </body>
15 </html>
```

On the right, the 'Instructions' panel contains the following text:

Prueba tú mismo:

Cambia float: left; a float: right;. ¿Dónde se posiciona la imagen ahora y cómo se envuelve el texto?

Elimina la propiedad float de la imagen. ¿Qué ocurre con la disposición de la imagen y el texto?

Elimina el overflow: hidden; del .contenedor-flotante y observa cómo el borde del contenedor ya no "encierra" la imagen flotante.

Prueba a flotar un elemento div en lugar de una imagen.

Explicación del Ejemplo:

Aplicamos **float: left;** a la imagen.

Esto la saca del flujo normal y la posiciona lo más a la izquierda posible dentro de su contenedor (.contenedor- flotante).

El texto del párrafo (<p>), que es un elemento de bloque que normalmente iría debajo de la imagen, ahora "sube" y se envuelve alrededor del lado derecho de la imagen.

Añadimos márgenes a la imagen (margin-right, margin- bottom) para evitar que el texto y la imagen estén pegados.

La propiedad **overflow: hidden;** en el contenedor es **una técnica común para hacer que el contenedor "contenga" los elementos flotantes**, asegurando que su borde o fondo se extienda para incluirlos.



CONSEJOS

- El valor predeterminado de float es none, lo que significa que el elemento seguirá el flujo normal de la página.
- La propiedad float puede usarse para diseños de página (aunque Flexbox/Grid son más modernos) y para hacer que el texto rodee imágenes.
- A menudo necesitarás usar la propiedad clear (o técnicas como overflow: hidden; en el contenedor) para evitar que elementos subsiguientes se coloquen junto al elemento flotante de forma indeseada.

Para más información, ver <http://www.w3schools.com/cssref/prclassfloat.asp>

Modelo de Caja Básico (height, margin, padding, width)



Las propiedades **height**, **margin**, **padding**, y **width** son la base del modelo de caja en CSS, determinando el tamaño de un elemento y el espacio a su alrededor.

Dominar estas cuatro propiedades te permite controlar el tamaño que ocupa cada elemento en la página y cómo interactúan espacialmente.



Las propiedades height y width pueden tener los siguientes valores:

- **auto**

Este es el valor predeterminado. El navegador calcula automáticamente la altura y el ancho del elemento en función de su contenido o contexto.

- **Sintaxis:** `div { width: auto; height: auto; }`
- Este valor es útil cuando el tamaño del elemento debe ajustarse automáticamente al contenido o contexto, como en una caja cuyo tamaño depende de la cantidad de texto o elementos dentro de ella.

- **length**

Define la altura o el ancho en unidades fijas, como **px** (píxeles), **cm** (centímetros), **em**, **rem**, entre otras.

- **Sintaxis:** `div { width: 300px; height: 150px; }`

- Se utiliza cuando se desea que el elemento tenga un tamaño fijo, sin importar el contenido dentro de él. Ideal para elementos que deben tener un tamaño específico y constante.

- **%** (Porcentaje)

Define la altura o el ancho en porcentaje respecto al bloque contenedor. Esto hace que el tamaño del elemento sea relativo al tamaño de su contenedor.

- **Sintaxis:** `div { width: 50%; height: 30%; }`

- Es muy útil en diseño **responsive** (adaptativo), donde se quiere que el tamaño de un elemento se ajuste dinámicamente en función del tamaño del contenedor.

- **initial**

Establece la propiedad height o width a su valor predeterminado, que normalmente es **auto**.

- **Sintaxis:** `div { width: initial; height: initial; }`

- Este valor es útil cuando se desea restablecer el valor de una propiedad a su estado predeterminado, sin tener que usar el valor explícito como auto.

- **inherit**

La propiedad height o width se hereda del valor de su elemento **padre**.

- **Sintaxis:** `div { width: inherit; height: inherit; }`

- Se usa cuando se desea que un elemento herede las propiedades de

tamaño de su contenedor o elemento padre. Esto es útil para mantener la consistencia en el diseño entre elementos relacionados.

La propiedad **height** **especifica la altura del área de contenido de un elemento**. Es decir, define la altura de la caja donde reside el contenido (texto, imágenes, etc.).

Es importante recordar que esta altura **NO incluye** el relleno (padding), el borde (border) ni el margen (margin) del elemento (a menos que uses box-sizing: border-box;, que es un tema más avanzado).

Puedes especificar la altura en unidades absolutas (como px), relativas (como %, vh, em), o dejar que el navegador la determine automáticamente.



The screenshot shows the trinket online code editor. The left pane displays the code for index.html and styles.css. The right pane shows the rendered result with instructions for experimentation.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10 <div class="caja-altura-fija">
11   Esta caja tiene una altura fija de 150px.
12 </div>
13
14 <div class="contenedor-imagen">
15   
16 </div>
17
18 </body>
19 </html>
```

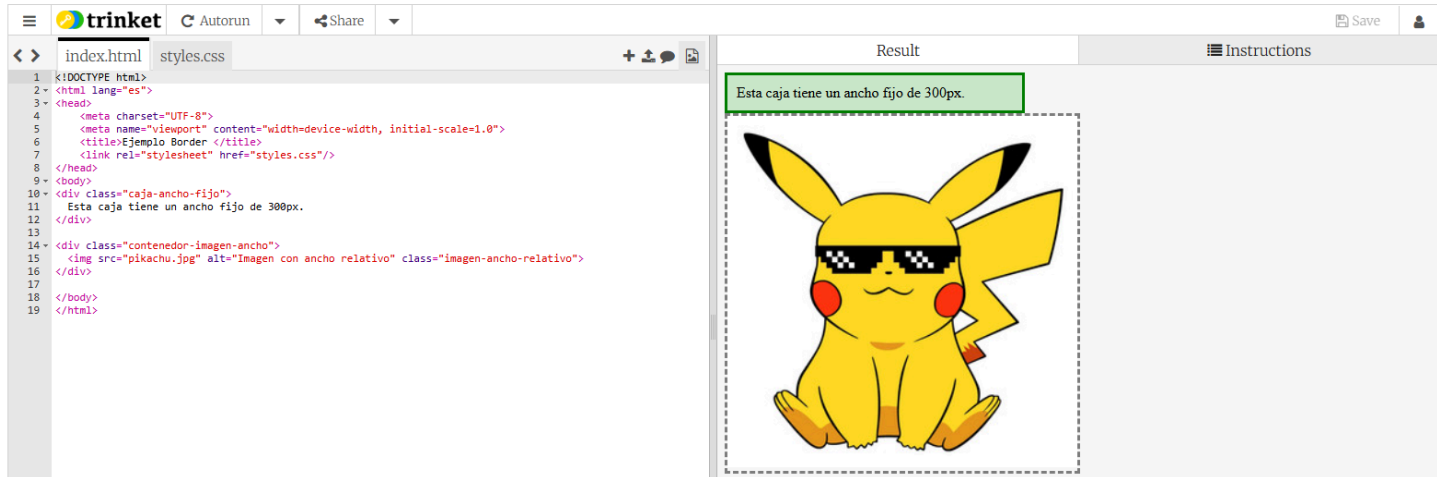
Prueba tú mismo:

- Cambia la unidad de height de la .caja-altura-fija a porcentaje (ej: height: 50%). Nota que un porcentaje de altura solo funciona si el contenedor padre tiene una altura definida.
- Cambia la altura de la imagen a un valor en vh (ej: height: 30vh;). Esto la hará 30% de la altura de la ventana (viewport).
- Elimina la propiedad height de uno de los elementos. ¿Qué ocurre (generalmente volverá al valor predeterminado auto)?
- Añade más texto al .caja-altura-fija para que su contenido sea más alto que 150px. ¿Qué ocurre con el texto que excede la altura definida? (Generalmente desbordará a menos que uses propiedades como overflow).

La propiedad **width** **especifica el ancho del área de contenido de un elemento**. Define la anchura de la caja donde reside el contenido.

Al igual que con height, esta anchura no incluye el relleno (padding), el borde (border) ni el margen (margin) del elemento (a menos que uses **box-sizing: border-box;**, que es un tema más avanzado).

Puedes especificar el ancho en unidades absolutas (como px), relativas (como %, vw, em), o dejar que el navegador lo determine automáticamente.



Explicación del Ejemplo:

- **.caja-ancho-fijo:** Su width se establece en 300px. El contenido se ajustará a este ancho. Fíjate en cómo el padding y el border *se añaden* a este ancho para determinar el ancho total que ocupa el elemento en la página (modelo de caja por defecto).
- **.imagen-ancho-relativo:** Su width se establece en 100%. Esto significa que su ancho será el 100% del ancho de su elemento *padre* (.contenedor-imagen). Como el contenedor tiene un ancho del 50% de su propio padre, la imagen tendrá un ancho igual al 50% del ancho del contenedor padre del .contenedor-imagen.

max-width:

Establece el ancho máximo que un elemento puede tener.

El elemento nunca será más ancho que el valor especificado por max-width, incluso si su width está configurado con un valor mayor o si su contenido intenta expandirlo más. Si su width es auto (por defecto para bloques) o un porcentaje, max-width limita hasta dónde puede crecer.

Es fundamental para el diseño responsivo. Evita que los elementos (como contenedores de contenido principal o imágenes) se vuelvan demasiado anchos en pantallas grandes, pero les permite encogerse y adaptarse a pantallas más pequeñas.

max-height

Establece la altura máxima que un elemento puede tener.

Similar a max-width, el elemento no crecerá verticalmente más allá de este valor. Si el contenido es mayor que la max-height, el contenido desbordará (el comportamiento por defecto) o se gestionará con overflow.

Uso común: Puede usarse para evitar que ciertos elementos ocupen demasiada altura en la página, forzando barras de desplazamiento si el contenido es extenso.

Resumen

En resumen, mientras que width y height intentan establecer una dimensión específica, max-width, min-width, max-height y min-height actúan como **restricciones** o **límites** a esas dimensiones. Son esenciales para crear diseños flexibles y responsivos que se adapten bien a diferentes tamaños de pantalla.



CONSEJOS

- La propiedad height por defecto es auto, lo que permite que el elemento ajuste su altura automáticamente para acomodar su contenido.
- La propiedad width por defecto es auto. Para elementos de bloque, auto significa ocupar el 100% del ancho disponible menos los márgenes. Para elementos en línea o flotados, auto significa ajustarse al ancho de su contenido.
- Las propiedades height y width no incluye el relleno, el borde ni el margen en el modelo de caja estándar.
- Usar porcentajes para height requiere que el elemento padre tenga una altura definida, de lo contrario, el porcentaje se calculará sobre una altura auto (lo que a menudo resulta en altura cero o comportamiento inesperado).
- Las unidades vh son útiles para alturas relativas a la ventana del navegador.
- Para más información, ver https://www.w3schools.com/css/css_dimension.asp

Propiedad: margin

```
margin-left: 12.5vw;  
margin-top: 100px;  
margin-bottom: 100px;  
margin-right: 40px;
```

La propiedad margin define el espacio transparente fuera del borde de un elemento. Este espacio empuja a otros elementos y crea separación entre ellos.

A diferencia del relleno (padding), el margen no tiene color de fondo ni está afectado por el borde. El margen es una propiedad abreviada que te permite establecer el margen para los cuatro lados a la vez.

Puedes usar un valor único (aplica a los 4 lados), dos valores (arriba/abajo, izquierda/derecha), tres valores (arriba, izquierda/derecha, abajo) o cuatro valores (arriba, derecha, abajo, izquierda - sentido horario).

SINTAXIS

/ Un valor: aplica a los 4 lados */* **margin: value;**

/ Dos valores: aplica (Top/Bottom) (Left/Right) */* **margin: value1 value2;**

/ Tres valores: aplica (Top) (Left/Right) (Bottom) */* **margin: value1 value2 value3;**

/ Cuatro valores: aplica (Top) (Right) (Bottom) (Left) */* **margin: value1 value2 value3 value4;**

/ Para un lado específico */*

- margin-top: value;
- margin-right: value;
- margin-bottom: value;
- margin-left: value;

trinket

Autorun

Share

Save

index.html

styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo Border </title>
7   <link rel="stylesheet" href="styles.css"/>
8 </head>
9 <body>
10 <p>Este es el primer párrafo.</p>
11 <p class="parrafo-con-margen">Este párrafo tiene margen.</p>
12 <div class="caja-con-margen">Esta caja tiene margen en todos sus lados.</div>
13
14 </body>
15 </html>
```

Result

Instructions

Este es el primer párrafo.

Este párrafo tiene margen.

Esta caja tiene margen en todos sus lados.



CONSEJOS

- Los márgenes crean espacio adicional alrededor de un elemento, mientras que el relleno (padding) crea espacio dentro del elemento.
- Cuando dos márgenes verticales adyacentes se tocan (por ejemplo, el margin-bottom de un párrafo y el margin-top del siguiente), a menudo colapsan, y el espacio resultante es el tamaño del margen más grande, no la suma de ambos (colapso de márgenes).
- Para más información, ver https://www.w3schools.com/css/css_boxmod



Propiedad: padding

La propiedad padding define el espacio transparente *dentro* del borde de un elemento, entre el contenido y el borde. Esencialmente, crea un "acolchado" alrededor del contenido. El background-color del elemento se extiende para cubrir el área de relleno.

El padding es una propiedad abreviada que te permite establecer el relleno para los cuatro lados a la vez.

Puedes usar un valor único (aplica a los 4 lados), dos valores (arriba/abajo, izquierda/derecha), tres valores (arriba, izquierda/derecha, abajo) o cuatro valores (arriba, derecha, abajo, izquierda - sentido horario).

SINTAXIS

`/* Un valor: aplica a los 4 lados */ padding: value;`

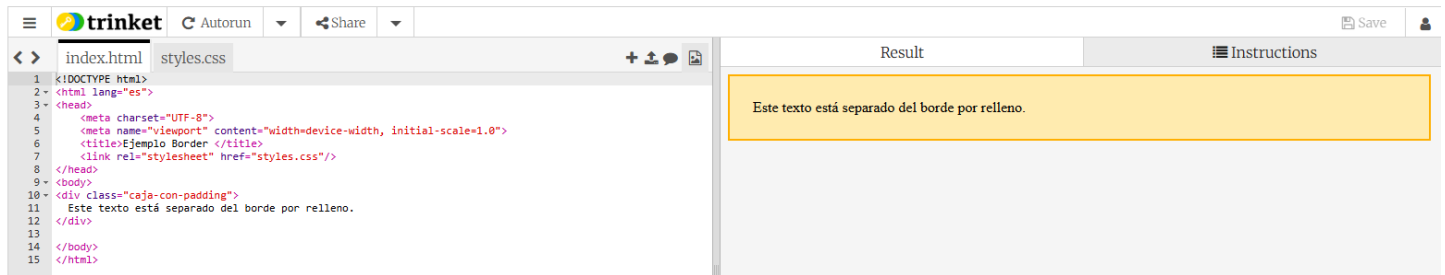
`/* Dos valores: aplica (Top/Bottom) (Left/Right) */ padding: value1 value2;`

`/* Tres valores: aplica (Top) (Left/Right) (Bottom) */ padding: value1 value2 value3;`

`/* Cuatro valores: aplica (Top) (Right) (Bottom) (Left) */ padding: value1 value2 value3 value4;`

`/* Para un lado específico */`

- `padding-top: value;`
- `padding-right: value;`
- `padding-bottom: value;`
- `padding-left: value;`



Explicación del Ejemplo:

Aplicamos un color de fondo y un borde a la caja para hacerla visible. Luego, padding: 25px; añade 25 píxeles de espacio transparente entre el texto "Este texto..." y el borde de la caja en los cuatro lados. El color de fondo amarillo claro se extiende para cubrir esta área de relleno.



CONSEJOS

- El valor predeterminado de padding es 0.
- Cuando se especifica un solo valor en la abreviada padding, se aplica el mismo relleno a los cuatro lados.
- El relleno forma parte del área sobre la cual se dibuja el background-color.

Para más información, ver
https://www.w3schools.com/css/css_boxmodel.asp

Centrando Contenido (Horizontalmente)

A menudo, no queremos que el contenido principal de nuestra página ocupe todo el ancho de la ventana, especialmente en pantallas grandes. Limitar su ancho y centrarlo mejora la legibilidad y la estética. La forma más común de lograr esto con elementos de **nivel de bloque** (como div, p, main, section) que tienen un ancho definido es utilizando la propiedad margin.

El Método Clave:

Para centrar un elemento de bloque que tiene un width o max-width definido:

- Asegúrate de que el elemento sea **display: block;** (la mayoría de los elementos contenedores lo son por defecto).
- Establece un ancho explícito usando **width o max-width.**
- Aplica margin-left: auto; y margin-right: auto;. La forma abreviada **margin: 0 auto;** establece el margen superior e inferior a 0 y le dice al navegador que distribuya el espacio restante en los lados izquierdo y derecho por igual, centrando el bloque.

Ejemplo Práctico: Un Contenedor Centrado

Aquí tienes un ejemplo sencillo con un contenedor div que centraremos en la página.

trinket

Autorun

Share

index.html

style.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Ejemplo de Centrado</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10
11   <div class="centered-container">
12     <h1>Contenido Centrado</h1>
13     <p>Este bloque de contenido tiene un ancho limitado y está centrado horizontalmente en la página.</p>
14     <p>Observa cómo se mantiene centrado si cambias el tamaño de la ventana del navegador (siempre que sea más ancho que el contenido).</p>
15
16     <div class="inline-elements">
17       <span>Elemento Inline 1</span>
18       <span>Elemento Inline 2</span>
19       <button type="button">Botón Inline-Block</button>
20     </div>
21
22   </div>
23
24 </body>
25 </html>
```

Result

Instructions

Contenido Centrado

Este bloque de contenido tiene un ancho limitado y está centrado horizontalmente en la página.

Observa cómo se mantiene centrado si cambias el tamaño de la ventana del navegador (siempre que sea más ancho que el contenido).



Elemento Inline 1 Elemento Inline 2 Botón Inline-Block

Pseudo-clases



Las pseudo-clases son selectores CSS especiales que te permiten aplicar estilos a elementos basados en un estado particular en el que se encuentran, en lugar de basarse únicamente en su estructura o atributos estáticos.

Son muy potentes para añadir interactividad visual y estilos dinámicos a tu página sin necesidad de JavaScript.

Algunos usos comunes incluyen estilizar enlaces en diferentes estados (visitado, no visitado, al pasar el ratón, activo), aplicar estilos al primer/último hijo de un elemento, o a elementos de un formulario en un estado específico (activo, deshabilitado).

SINTAXIS

```
selector:pseudo-class {  
  
  propiedad: value;  
  
}
```

Enlace no visitado:

```
a:link {  
  
color: blue;  
  
}
```

Enlace al pasar el ratón:

```
a:hover {  
  
color: green;  
  
}
```

Las pseudo-clases se usan para agregar efectos especiales a algunos selectores. Se utilizan principalmente para:

- Estilizar un elemento cuando el usuario pasa el ratón sobre él.
- Estilizar enlaces visitados y no visitados de manera diferente.



Explicación del Ejemplo:

- **a:link:** Selecciona todos los elementos <a> (enlaces) cuyo destino (href) aún no ha sido visitado por el usuario. Les aplica color azul y quita el subrayado.
- **a:visited:** Selecciona todos los elementos <a> cuyo destino (href) ya ha sido visitado por el usuario. Les aplica color púrpura.
- **a:hover:** Selecciona elementos cuando el puntero del ratón está sobre ellos. Les aplica color verde y añade un subrayado.
- **a:active:** Selecciona elementos mientras se está haciendo clic activamente en ellos (desde el momento en que se presiona el botón del ratón hasta que se suelta). Les aplica color rojo.

El orden de estas pseudo-clases para enlaces importa debido a la especificidad y el estado: a menudo se recomienda el orden **Link Visited Hover Active (LVHA)** para que los estilos se apliquen correctamente.



CONSEJOS

- Las pseudo-clases se usan para agregar efectos especiales o estilos basados en el estado o la posición de los elementos.
- Se utilizan principalmente para estilizar enlaces en diferentes estados (:link, :visited, :hover, :active) y para seleccionar elementos basados en su relación con sus hermanos o padre (:first-child, :last-child, :nth-child).
- Para más información, ver https://www.w3schools.com/css/css_pseudo_classes.asp

Construyendo Menús de Navegación



Una navegación bien diseñada es la columna vertebral de cualquier sitio web exitoso. Permite a los visitantes encontrar lo que buscan de manera rápida y sencilla, mejorando significativamente la experiencia del usuario.

En el corazón de la navegación web se encuentra la humilde lista de enlaces HTML. Sin embargo, es a través del poder de CSS que esta lista básica se transforma en una barra de navegación atractiva y funcional, capaz de adaptarse a diferentes diseños y comportamientos o adhesivas.

1

Esqueleto: Estructura HTML Fundamental

Antes de aplicar cualquier estilo, necesitamos una base sólida en HTML. La práctica recomendada es utilizar elementos semánticos que representen la estructura de un menú de navegación de manera clara.

Una lista desordenada () es ideal para agrupar los enlaces, donde cada ítem de la lista () contiene un enlace (<a>) que apunta a la ubicación deseada.

The screenshot shows the Trinket.io web editor. On the left, the code editor displays the following HTML code for 'index.html':

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Base Menu HTML</title>
7
8 </head>
9 <body>
10
11 <h1>Estructura Base de un Menú de Navegación</h1>
12 <p>Este es el HTML sin aplicar estilos CSS.</p>
13
14 <nav>
15   <ul>
16     <li><a href="#inicio">Inicio</a></li>
17     <li><a href="#noticias">Noticias</a></li>
18     <li><a href="#contacto">Contacto</a></li>
19     <li><a href="#acerca-de">Acerca de</a></li>
20   </ul>
21 </nav>
22
23 <p>Observa la lista básica con viñetas por defecto.</p>
24
25 </body>
26 </html>
```

On the right, the 'Result' tab shows the rendered output. It features the title 'Estructura Base de un Menú de Navegación' and the text 'Este es el HTML sin aplicar estilos CSS.' Below this, there is a bulleted list of links: 'Inicio', 'Noticias', 'Contacto', and 'Acerca de'. A note at the bottom states: 'Observa la lista básica con viñetas por defecto.'

<https://trinket.io/embed/html/cb2829eb4abd>

Explicación:

El código muestra la estructura HTML esencial: un contenedor `<nav>` que envuelve una lista `<ul>`. Cada ítem `<li>` contiene un enlace `<a>`.

Esta estructura es simple, semántica y accesible, proporcionando la base perfecta para aplicar estilos con CSS.

Sintaxis y Conceptos:

- `<nav>`: Contenedor semántico para bloques de navegación.
- `<ul>`: Lista desordenada.
- `<li>`: Ítem de lista.
- `<a>`: Enlace con atributo href.

2

Lienzo Limpio: Reseteando Estilos por Defecto

Los navegadores aplican estilos predeterminados a los elementos HTML, y las listas no son una excepción.

Suelen tener viñetas (marcadores de lista), así como cierto margen y relleno. Para tener control total sobre la apariencia de nuestra barra de navegación, es fundamental eliminar estos estilos por defecto utilizando las propiedades `list-style-type`, `margin` y `padding`.

Ejemplo: Eliminando Marcadores y Espaciado Inicial

The screenshot shows the Trinket.io web application. On the left, the code editor displays the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link rel="stylesheet" href="styles.css">
7   <title>Base Menu HTML</title>
8
9 </head>
10 <body>
11
12 <h1>Estructura Base de un Menú de Navegación</h1>
13 <p>Este es el HTML con estilos CSS.</p>
14
15 <nav>
16   <ul>
17     <li><a href="#inicio">Inicio</a></li>
18     <li><a href="#noticias">Noticias</a></li>
19     <li><a href="#contacto">Contacto</a></li>
20     <li><a href="#acerca-de">Acerca de</a></li>
21   </ul>
22 </nav>
23
24 <p>Observa la lista básica ya SIN viñetas por defecto.</p>
25
26 </body>
27 </html>
```

On the right, the 'Result' tab shows the rendered output. The title is 'Estructura Base de un Menú de Navegación'. Below the title, it says 'Este es el HTML con estilos CSS.' and displays a navigation menu with four links: 'Inicio', 'Noticias', 'Contacto', and 'Acerca de'. Below the menu, it says 'Observa la lista básica ya SIN viñetas por defecto.'

<https://trinket.io/embed/html/9f7f38449d62>

Explicación:

El aplicar **list-style-type: none;**, **margin: 0;** y **padding: 0;** al elemento `ul`, eliminamos los marcadores de lista y el espaciado predeterminado del navegador. La lista ahora se presenta de forma compacta, lista para ser transformada en una barra de navegación vertical u horizontal mediante la disposición de sus elementos `li` y `a`.

Estilo del menú: Línea 28 del archivo `styles.css`

Sintaxis y Consejos:

- `list-style-type: none;`: La forma estándar de quitar viñetas/marcadores.
- `margin: 0;` y `padding: 0;`: Usados en muchos resets para eliminar el espaciado por defecto de los elementos.

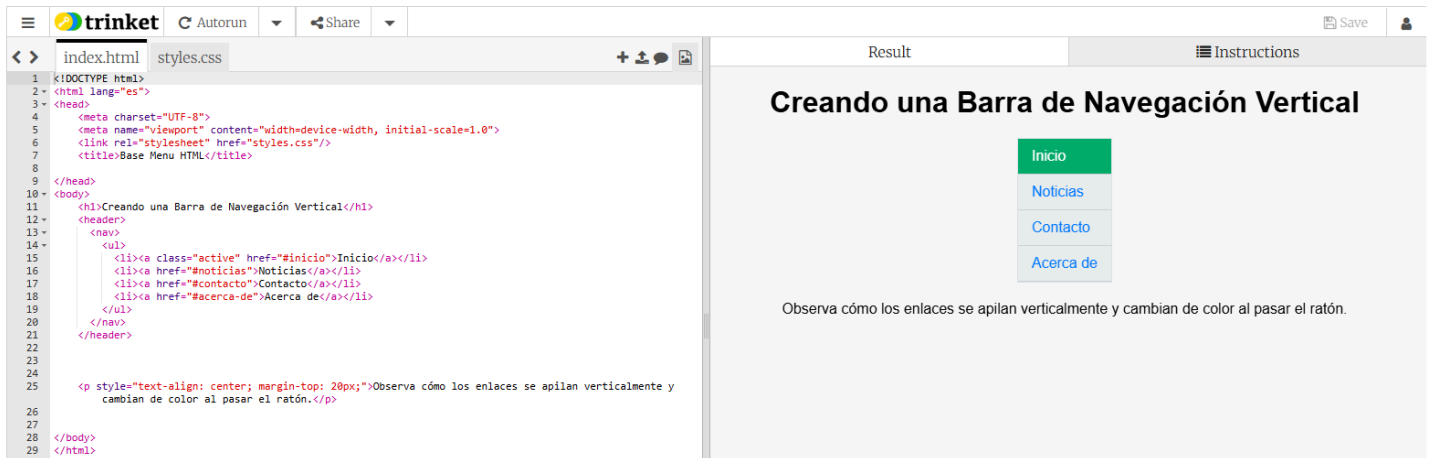
3



Una barra de navegación **vertical** apila los enlaces uno debajo del otro.

Este diseño es ideal para barras laterales, menús desplegables o listas de navegación en el pie de página.

El paso clave es hacer que los enlaces (`<a>`), que son elementos en línea por defecto, se comporten como elementos de bloque.



<https://trinket.io/embed/html/ab3f6a3103c2>

Explicación del Ejemplo:

La clave para la **navegación vertical es display: block;** aplicada a los enlaces (li a). Esto hace que cada enlace ocupe todo el ancho disponible en su contenedor y fuerce un salto de línea antes y después, resultando en una pila vertical.

El **width aplicado al ul** define el ancho total de esta columna de enlaces.

El padding en los enlaces crea el área "clickeable" y añade espacio alrededor del texto. Las propiedades de color, background-color, text-decoration y la pseudo-clase :hover se utilizan para dar estilo visual y efectos interactivos.

La clase .active demuestra cómo resaltar el enlace de la página actual.

Sintaxis y Consejos:

display: block; Convierte un elemento en línea en un bloque, permitiendo controlar su width y height y forzando saltos de línea.

width: Define el ancho del área de contenido.

Menú de Navegación:

Horizontal

Una barra de navegación horizontal muestra los enlaces uno junto al otro. Una técnica común y efectiva para lograr esto es usando la propiedad **float** en los elementos de lista (), sacándolos del flujo normal para que se alineen horizontalmente.

Ejemplo: Diseño de una Barra Horizontal con Float

Autosave
Share

index.html
styles.css

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link rel="stylesheet" href="styles.css">
7   <title>Base Menu HTML</title>
8 </head>
9 <body>
10   <h1>Creando una Barra de Navegación Horizontal (Float)</h1>
11
12   <header>
13     <nav>
14       <ul class="clearfix">
15         <li><a class="active" href="#inicio">Inicio</a></li>
16         <li><a href="#noticias">Noticias</a></li>
17         <li><a href="#contacto">Contacto</a></li>
18         <li><a href="#acerca-de">Acerca de</a></li>
19       </ul>
20     </nav>
21   </header>
22
23   <p style="margin-top: 20px;">Observa cómo los enlaces se alinean horizontalmente. El <code>overflow: hidden;</code> en el UL (o una técnica <code>clearfix</code> como la usada aquí) asegura que el fondo y borde del UL contengan los elementos flotantes.</p>
24
25
26
27 </body>
28 </html>

```

Result
Instructions

Creando una Barra de Navegación Horizontal (Float)

Inicio
Noticias
Contacto
Acerca de

Observa cómo los enlaces se alinean horizontalmente. El `overflow: hidden;` en el UL (o una técnica `clearfix` como la usada aquí) asegura que el fondo y borde del UL contengan los elementos flotantes.

<https://trinket.io/embed/html/8e5370356e39>

Menú de Navegación:

Fijos o Adhesivos

Para mantener una barra de navegación **siempre visible mientras el usuario se desplaza**, se utiliza la propiedad position.

Los valores fixed o sticky logran este efecto, fijando el elemento a la ventana del navegador o a su contenedor en un punto de scroll determinado.

Ejemplo: Barra de Navegación Fija en la Parte Superior

The image shows a Trinket.io code editor interface. On the left, the code editor displays the following HTML and CSS code:

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link rel="stylesheet" href="styles.css">
7   <title>Base Menu HTML</title>
8 </head>
9 <body>
10 <h1>Barra de Navegación Fija Superior</h1>
11 <p>Desplázate hacia abajo para ver que la barra de navegación se mantiene en la parte superior de la ventana.</p>
12 <div>
13   <div>
14     <div>
15       <div>
16         <div>
17           <div>
18             <div>
19               <div>
20                 <div>
21                   <div>
22                     <div>
23                       <div>
24                         <div>
25                           <div>
26                             <div>
27                               <div>
28                                 <div>
29                                   <div>
30                                     <div>
31                                       <div>
32                                         <div>
33                                           <div>
34                                             <div>
35                                             </div>
36                                           </div>
37                                         </div>
38                                       </div>
39                                     </div>
35                                     </div>
36                                   </div>
37                                 </div>
38                               </div>
39                             </div>
40                           </div>
41                         </div>
42                       </div>
43                     </div>
44                   </div>
45                 </div>
46               </div>
47             </div>
48           </div>
49         </div>
50       </div>
51     </div>
52   </div>
53 </div>
54 </body>
55 </html>
  
```

The CSS file (styles.css) is empty. The result preview on the right shows a web page with a fixed navigation bar at the top containing links: Inicio, Noticias, Contacto, and Acerca de. Below the navigation bar, there is a large light blue area with the text "Contenido de ejemplo para generar scroll." and a vertical scrollbar on the right side.

<https://trinket.io/embed/html/9e7819108461>

Construyendo Galería de imágenes



Ceibal

Una galería de imágenes organizada es esencial para sitios web que muestran portafolios, productos o colecciones visuales. CSS nos permite transformar una simple lista de imágenes en una presentación estructurada.

Esta guía te llevará a través del proceso de construcción de una galería básica con lo visto en el curso.

1

trinket

Autorun

Share

Save

index.html

styles.css

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Galería de Imágenes Estilizada (Local)</title>
7   <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10
11   <h1>Mi Galería de Imágenes Estilizada con CSS</h1>
12
13   <div class="gallery-container">
14     <div class="gallery">
15       
16       <div class="desc">Un ratón eléctrico</div>
17     </div>
18
19     <div class="gallery">
20       
21       <div class="desc">Un monstruo adorable</div>
22     </div>
23
24     <div class="gallery">
25       
26       <div class="desc">Un pequeño dragón de fuego</div>
27     </div>
28
29     <div class="gallery">
30       
31       <div class="desc">El logo de los Minions</div>
32     </div>
33
34     <div class="gallery">
35       
36       <div class="desc">Un personaje de arte</div>
37     </div>
38
39     <div class="gallery">
40       
41       <div class="desc">Imagen de ejemplo 6</div>
42     </div>
43
44     <div class="gallery">
45       
46     </div>
47   </div>
48 </body>
49 </html>
```

Result

Instructions

Mi Galería de Imágenes Estilizada con CSS



<https://trinket.io/embed/html/c7971ce6a5ec>

Explicación:

1. Contenedor Individual ([div.gallery](#)):

- Comenzamos dando a cada contenedor de imagen ([div.gallery](#)) un **border** para que podamos ver sus límites, un **margin** para separarlos entre sí, y un **width** para controlar el ancho de cada caja individual. Esto define el tamaño y la estructura básica de cada "elemento" de la galería.
- Luego, aplicamos **float: left;** a [div.gallery](#). Esta es la propiedad clave para el layout. Indica a cada caja que se mueva lo más a la izquierda posible. Los elementos que le siguen harán lo mismo, colocándose a su derecha si hay espacio. Cuando una línea se llena, el siguiente elemento flotante "cae" a la línea siguiente. Así se forma la cuadrícula.

2. La Imagen ([div.gallery img](#)):

- Para que la imagen siempre encaje dentro de la caja [div.gallery](#) sin desbordarse, le damos un **width: 100%;**. Esto significa que la imagen ocupará exactamente el mismo ancho que su contenedor padre ([div.gallery](#)).
- Aplicamos **height: auto;** a la imagen. Esto es vital. Indica que la altura de la imagen debe calcularse automáticamente basándose en el ancho establecido (100%) para mantener la proporción original de la imagen. Si no usaras **height: auto;** y la imagen original tuviera una proporción diferente a la de la miniatura deseada, podría distorsionarse.
- **display: block;** en la imagen es a menudo útil junto con **width: 100%** para asegurar que ocupe su propia línea dentro de su contenedor y que el 100% del ancho funcione de manera predecible.

3. La Descripción ([div.desc](#)):

- Aplicamos **padding: 15px;** al [div.desc](#) para crear un espacio interno de 15 píxeles alrededor del texto de la descripción, evitando que el texto se pegue al borde de la caja de descripción (que heredaría el borde de la caja [div.gallery](#)).
- Usamos **text-align: center;** para centrar el texto de la descripción horizontalmente dentro de esta área.

4. Interactividad con el Ratón ([div.gallery: hover](#)):

- Aplicamos estilos a la pseudo-clase **:hover** en [div.gallery](#). Esto nos permite definir cómo se ve la caja cuando el usuario pasa el puntero del ratón sobre ella. En este caso, cambiamos el **border** a un color más oscuro y añadimos una sombra ([box-shadow](#)) para dar retroalimentación visual.

5. El Contenedor de la Galería (.gallery-container) - Limpiando Floats:

- Cuando usas float, los elementos flotantes se sacan parcialmente del flujo normal. Esto puede causar que el contenedor padre no "encierre" completamente la altura de los elementos flotantes, o que el contenido que viene después del contenedor se posicione incorrectamente.
- La propiedad **overflow: hidden;** aplicada al contenedor padre (.gallery-container) es una técnica común y sencilla para resolver esto. Fuerza al contenedor a ajustarse a sus hijos flotantes, asegurando que su fondo, borde o el siguiente contenido en la página se comporten como se espera. (Otra técnica es usar un elemento vacío con clear: both; o pseudoelementos ::after con clear: both;).

Construyendo un Formulario con estilo



Ceibal

The screenshot shows the Trinket.io web editor interface. On the left, the code editor displays the HTML and CSS for a contact form. The HTML code includes a head section with meta tags for charset, viewport, and title, and a body section with a form container. The CSS code defines the form container and form groups. The right side of the editor shows a preview of the rendered form, which is titled 'Contáctanos' and includes input fields for 'Nombre:', 'Correo electrónico:', and 'Mensaje:', along with a 'Enviar Mensaje' button.

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Formulario Estilizado</title>
7   <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10
11   <div class="form-container">
12     <h2>Contáctanos</h2>
13     <p>Llena el formulario para ponerte en contacto.</p>
14     <form>
15       <div class="form-group">
16         <label for="nombre">Nombre:</label>
17         <input type="text" id="nombre" name="nombre" required>
18       </div>
19
20       <div class="form-group">
21         <label for="email">Correo electrónico:</label>
22         <input type="email" id="email" name="email" required>
23       </div>
24
25       <div class="form-group">
26         <label for="mensaje">Mensaje:</label>
27         <textarea id="mensaje" name="mensaje" rows="5" required></textarea>
28       </div>
29
30       <button type="submit" class="submit-button">Enviar Mensaje</button>
31     </form>
32   </div>
33
34 </body>
35 </html>

```

<https://trinket.io/embed/html/448a97665628>

Explicación:

Centrado y Contenedor:

- Se usó CSS en el body (con display: flex y propiedades de alineación) y en el contenedor principal del formulario (.form-container con max-width y margin: auto o por el centrado flex) para **centrar el formulario en la página** y darle un área definida.
- El .form-container recibió un **fondo blanco, padding** (espacio interno) para que el contenido no toque los bordes, **bordes redondeados** (border-radius) y una **sombra** (box-shadow) para que visualmente destaque.

Organización de Campos (Apilado):

- Se aplicó **display: block;** a las etiquetas (label) y a los campos de entrada (input, textarea). Esto es clave para que cada etiqueta y su campo asociado se muestren **uno debajo del otro**, en lugar de intentar alinearse en la misma línea.
- Se usó **margin-bottom** en los .form-group (los divs que envuelven cada par label-input) para crear espacio **entre los diferentes campos** del formulario.

Estilo de Campos de Entrada:

- Los campos de texto y el área de texto se configuraron con **width: 100%;** para que ocupen todo el ancho disponible de su contenedor padre.
- La propiedad **box-sizing: border-box;** se usó en los campos para asegurar que el **padding** (espacio interno) y el **border** (borde) que les aplicamos no se *sumen* al ancho total, sino que se incluyan dentro de ese width: 100%. Esto ayuda a que no desborden.
- Se les dio **padding, border y border-radius** para definir su apariencia visual y añadir espacio interno para escribir cómodamente.
- Se usó la pseudo-clase **:focus** para aplicar un estilo visual (cambio de borde, sombra) **solo cuando el usuario selecciona** un campo para escribir.

Estilo del Botón:

- El botón de envío (.submit-button) también se configuró con **display: block;** y **width: 100%;** para que sea un elemento de acción claro y grande.
- Se definió su **background-color** y **color** (texto) para un contraste llamativo.
- Se añadió **padding** para darle un tamaño adecuado, se eliminó el borde por defecto (**border: none;**) y se aplicaron **border-radius**.
- Se usaron las pseudo-clases **:hover, :active,** y **:focus** para cambiar su apariencia **cuando el usuario pasa el ratón, hace clic o lo selecciona**, proporcionando retroalimentación visual y una mejor experiencia de usuario.

